

Competitive Programming Notes

Last updated: September 8, 2026

Abstract

This document is a personal collection of competitive programming problems solved over time. Each entry includes a brief restatement of the problem, the reasoning process that led to the solution, and a final code implementation. Problems are presented in chronological order, without thematic categorization, with the goal of tracking technical progress and consolidating problem-solving skills. I hope that this material may be useful to others as well, and recommends consulting the provided solutions only after making a genuine attempt at solving the problems independently. The ratings are a mix of CF rating, personal experience and CLIST rating which mirrors perfectly the general real time performance of contestants.

Template

When reading the implementation keep in mind that above the solve function i always use these two CP-templates one for java and for c++.

C++

```
#include <bits/stdc++.h>
using namespace std;

// debug
#define debug(x) cerr << #x << " = " << x << '\n';
#define vdebug(a, n) cerr << #a << " = "; for(int i = 0; i < n; i++) cerr << a[i] << " \n"[i == n - 1];
#define mdebug(a, n, m) cerr << #a << "\n"; for(int i = 0; i < n; i++) for(int j = 0; j < m; j++) cerr << a[i][j] << " \n"[j == m - 1];

// utilities iterators
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
#define repz(i, n) for(int i = 0; i < (n); i += 1)
#define repo(i, n) for(int i = 1; i <= (n); i += 1)
#define sz(a) ((int)(a.size()))

// #define int long long (overflow issue)
// generic macros
#define F first
#define S second
#define und -1
#define pb push_back
#define ppb pop_back

// common type shortening
typedef vector<int> vi;
typedef vector<long long> vl;
typedef vector<bool> vb;
typedef long long ll;
typedef pair<int, int> pii;
typedef tuple<int, int, int> tii;

// hash seed gen
mt19937 rnd(chrono::steady_clock::now().time_since_epoch().count());
```

```
// useful constants
const int mod = 998244353;
const ll Binf = LLONG_MAX;
const ll Bminf = LLONG_MIN;
const int Sinf = INT_MAX;
const int Sminf = INT_MIN;
const int nxm = 2 * 1e5 + 10;
const int mx1 = 1000 + 10;
const int lnxm = 21;
```

Java

```
import java.util.*;
import java.io.*;

public class Main {
    final static int ArrPadd = 10;
    static class FastRead {
        StringTokenizer s;
        BufferedReader bf;
        public FastRead() {
            bf = new BufferedReader(new InputStreamReader(System.in));
        }
        String next() {
            while(s == null || !s.hasMoreElements()) {
                try {
                    s = new StringTokenizer(bf.readLine());
                } catch (IOException e) {
                    e.printStackTrace();
                }
            }
            return s.nextToken();
        }
        int nextInt() {
            return Integer.parseInt(next());
        }
        long nextLong() {
            return Long.parseLong(next());
        }
        double nextDouble() {
            return Double.parseDouble(next());
        }
        String nextLine() {
            String str = "";
            try {
                if(s.hasMoreTokens())
                    str = s.nextToken();
                else
                    str = bf.readLine();
            } catch (IOException e) {
                e.printStackTrace();
            }
            return str;
        }
    }
    // main method
}
```

Monopati

Source: Codeforces Round 1063 (1500)

Link: <https://codeforces.com/contest/2163/problem/C>

Problem statement Given a $2 \times n$ matrix A where $1 \leq a_{i,j} \leq 2n$, define $F = f(l, r, A)$ a boolean grid where $f_{ij} = [l \leq a_{ij} \leq r]$. Count all the possible pairs (l, r) such that there exist a down-left path from $f_{11} \rightarrow f_{2n}$ (Each element that forms the path has to be 1).

Reasoning Note that for a pair (l, r) to be counted, $F = f(l, r, A)$ have the following form:

$$\begin{bmatrix} 1 & \cdot & 1 & 1 & * & * \\ * & \cdot & * & 1 & \cdot & 1 \end{bmatrix}$$

More formally there should be at least a path, such path is composed by a prefix of indices of the top row, and a suffix of indices of the bottom row, such that all elements in the prefix and suffix satisfy the aforementioned property. Moreover the prefix and the suffix should be connected (more formally given $P(i) = [1, \dots, i]$ and $S(j) = [j, \dots, n]$, then $P(i)$ and $S(j)$ are connected if and only if $i \leq j$). Let's look at the two following possible matrices

$$F_1 = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix} \quad F_2 = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{bmatrix}$$

We can observe that the pair (l_2, r_2) has to be tighter than (l_1, r_1) that means that if we relax the second pair, we are gonna get the pair that generate F_1 . More in general we will look at all possible tight paths $p_i = [P(i), S(i)]$ and we'll find the pair (l, r) such that all elements $l \leq a_{p_i} \leq r$ and any increment of l or decrement of r will make the path not valid. For each l we save what is the minimal r such that $f(l, r)$ contains a valid path. We know that if there was a valid pair (l_1, r_1) then given (l_2, r_2) with $l_2 < l_1$ i can make at most $2n - \min(r_2, r_1) + 1$ pairs out of l_2 ;

```
int t[nxm], b[nxm];
int mnl[nxm], mnr[nxm];
int mxl[nxm], mxr[nxm];
int n;
void solve(){
    cin >> n;
    for(int i = 0; i < n; i++){
        cin >> t[i];
    }
    for(int i = 0; i < n; i++){
        cin >> b[i];
    }

    mnl[0] = t[0]; mxl[0] = t[0];
    for(int i = 1; i < n; i++){
        mxl[i] = max(mxl[i - 1], t[i]);
        mnl[i] = min(mnl[i - 1], t[i]);
    }
    mnr[n - 1] = b[n - 1]; mxr[n - 1] = b[n - 1];
    for(int i = n - 2; i >= 0; i--){
        mxr[i] = max(mxr[i + 1], b[i]);
        mnr[i] = min(mnr[i + 1], b[i]);
    }
    vi l(2 * n + 1, 2 * n + 1);
    for(int i = 0; i < n; i++){
        int lb = min(mnl[i], mnr[i]);
        l[lb] = min(l[lb], max(mxr[i], mxl[i]));
    }
    ll tot = 0;
    int mini = 2 * n + 1;
    for(int lb = 2 * n; lb >= 1; lb--){

```

```

    mini = min(l[lb], mini);
    tot += max(2 * n - mini + 1, 0);
}
cout << tot << '\n';
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int tt;
    cin >> tt;
    while(tt--){
        solve();
    }
    return 0;
}

```

Complexity $O(n)$

Make Connected

Source: Pinely Round 5 (1700)

Link: <https://codeforces.com/contest/2161/problem/B>

Problem Statement Given an integer n and an $n \times n$ grid consisting only of characters $\#$ and $\cdot$. Determine whether it is possible to make all cells connected by optionally changing any $\cdot$ cell into $\#$. However, a configuration is considered invalid if any row or any column contains more than two $\#$ in a row.

$$\begin{bmatrix} \# & \# & \# \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

Note that it's impossible to change any $\#$ cell, so this configuration it's invalid and it will, under any transformation, be invalid.

Reasoning If there are no $\#$ then by definition the property is satisfied, assume now there is at least one $\#$ and take the highest one.

$$\begin{bmatrix} \cdot & \# & \cdot \\ \cdot & \cdot & \# \\ \cdot & \# & \cdot \end{bmatrix}$$

in this case from there i can zig zag to the bottom in 4 different possible patterns $(d, r), (r, d), (l, d), (d, l)$. Observe that my zig zag can never switch direction or pattern, since as soon as i change 3 $\#$ in a row will form. For example $d-l-(d-l-r-d), d-l-(d-l-d-r), d-l-(d-l-l-d), d-l-d-l-d-l$, so the only way to not crate an invalid conncted area is to always reapeat the same pattern. So we can start from the top spot and try all the possible zig zag pattern starting from there and ending as soon as i go off bounds (i can never turn horizontally or vertically). If my zig zag pattern contains all the $\#$ in the original grid then it is possible.

Edge Cases It's possible to form a 2×2 square without breaking the bounds, so consider it as a possibility. Assuming the starting spot is top-leftmost $\#$ cell, we know certainly that the square has to be constructed on the right side of it, so check only that construction. If we start from the top-leftmost $\#$ cell and another $\#$ is adjacent to it on the right start then th $(l-d)$ zig zag has to start from the rightmost cell, so that the zig zag will include both cells.

```

void solve(){
    int n;
    cin >> n;
    char g[100][100], a[100][100], b[100][100];

```

```

for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        cin >> g[i][j];
int f_r = -1;
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        if(g[i][j] == '#' && f_r == -1)
            f_r = i;
if(f_r == -1){
    cout << "Yes" << '\n';
    return;
}
pii f_b_l, f_b_r;
for(int j = 0; j < n; j++)
    if(g[f_r][j] == '#'){
        f_b_l = {f_r, j};
        break;
    }
for(int j = n - 1; j >= 0; j--){
    if(g[f_r][j] == '#'){
        f_b_r = {f_r, j};
        break;
    }
}

function<bool(void)> check = [&]() {
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n; j++){
            if(g[i][j] == '#')
                if(b[i][j] == '.')
                    return false;
        }
    }
    return true;
};

function<bool(pii, pii, pii)> build = [&](pii h, pii k, pii f_b){
    memset(b, '.', sizeof(b));
    pii s = f_b;
    while(!(s.F >= n || s.S >= n || s.F < 0 || s.S < 0)){
        b[s.F][s.S] = '#';
        s.F += h.F;
        s.S += h.S;
        if(s.F >= n || s.S >= n || s.F < 0 || s.S < 0)
            break;
        b[s.F][s.S] = '#';
        s.F += k.F;
        s.S += k.S;
    }
    return check();
};

function<bool(pii)> square = [&](pii f_b){
    memset(b, '.', sizeof(b));
    if(f_b.F != n - 1 && f_b.S != n - 1){
        b[f_b.F][f_b.S] = '#';
        b[f_b.F + 1][f_b.S + 1] = '#';
        b[f_b.F][f_b.S + 1] = '#';
        b[f_b.F + 1][f_b.S] = '#';
        return check();
    }
}

```

```

    }
    return false;
};
bool chk1, chk2, chk3, chk4, chk5;
chk1 = build({1, 0}, {0, -1}, f_b_r);
chk2 = build({0, -1}, {1, 0}, f_b_r);
chk3 = build({1, 0}, {0, 1}, f_b_l);
chk4 = build({0, 1}, {1, 0}, f_b_l);
chk5 = square(f_b_l);
cout << (chk1 || chk2 || chk3 || chk4 || chk5 ? "yes\n" : "no\n");
}

signed main(){
    ios::sync_with_stdio(false);
    cin.tie(0);
    int tt;
    cin >> tt;
    while(tt--){
        solve();
    }
    return 0;
}

```

Complexity $O(n^2)$

Triple Removal

Source: Squarepoint challenge (1400)

Link: <https://codeforces.com/contest/2152/problem/C>

Problem Statement Given an array a of n elements such that a_i is either 0 or 1, and q queries characterized by integers l and r . triple-deletion operation is defined as follows. Choose three indices i, j, k such that $1 \leq i < j < k \leq n$, and delete the elements a_i, a_j, a_k from the array. The cost of this operation is defined as $\min(j-i, k-j)$, that is, the minimum of the distances between consecutive chosen indices. Define the triple-deletion process of a subarray as a series of operation made on the subarray such that at the end of the aforementioned process the subarray will be empty. For each subarray queried print the total cost of the minimum cost triple-deletion process.

Reasoning Observe that at each operation we have to delete 3 ones or 3 zeroes, so for any process to come to an end there should be a number of ones and a number of zeroes that are both divisible by 3. If a queried subarray $S(l, r)$ doesn't satisfy this property then there's no process, otherwise there is. Observe that if we delete the ones before the zeroes then the cost to remove the ones is just $n_o[l, r]/3$. The opposite has to be true too. Given that an array a satisfies the property then the first optimal operation has cost of at most 3 and all the others have cost of 1. We will prove that by contradiction, suppose that there exists an array a composed of n elements and suppose that the minimum distance between any two 1 or any two 0 elements is > 2 , notice that if such minimum distance is between two ones, then two zeroes elements with distance of 1 should exist, so this is contradictory. This means that the only cases where the first move has a cost of 2 instead of 1 are the arrays $[1, 0, 1, 0, \dots, 1, 0]$ and $[0, 1, 0, 1, \dots, 0, 1]$. Observe at last that given any valid array that is not "special" can be processed with cost $n_o[a]/3 + n_z[a]/3$ and that is the minimum possible. The second statement is trivially true. The first statement is true if we can always keep an array "not special". By deleting one group first and the other successively then the array can never be special and thus the operations can never have cost > 1 .

```

int n, q, a[250001];
int p_o[250001][{}], chk[250001][{}];
void solve(){
    cin >> n >> q;

```

```

int n_z = 0, n_o = 0;
repo(i, n)
    cin >> a[i];
repo(i, n)
    p_o[i] = p_o[i - 1] + a[i];
repo(i, n)
    chk[i] = INT_MAX;
int b = 1;
repo(i, n){
    if(i > 1 && a[i] == (a[i - 1] + 1) % 2){
        chk[i] = b + ((i - b + 1) & 1);
        continue;
    }
    b = i;
}
repo(i, q){
    int l, r, cnt_o, cnt_z;
    cin >> l >> r;
    cnt_o = p_o[r] - p_o[l - 1];
    cnt_z = r - l + 1 - cnt_o;
    if(cnt_o % 3 != 0 || cnt_z % 3 != 0){
        cout << -1 << '\n';
        continue;
    }
    cout << cnt_o / 3 + cnt_z / 3 + (chk[r] <= 1) << '\n';
}
}

```

Complexity $O(n)$

War Strategy

Source: Hello 2026 (1500)

Link: <https://codeforces.com/contest/2183/problem/C>

Problem Statement Given an array a of n elements ($a = a_1, \dots, a_n$, m days and an index $1 \leq k \leq n$. At day 0 there is a soldier in position k , each day d after the zero-th day, the commander can choose an index $i \in [1, n]$ and move any number of soldiers from i to $i + 1$ or from i to $i - 1$ (he can't move soldiers left from position 1 and right from position n), at the end of d a new soldier appears at position k . Determine what is the maximum number of fortified positions after the end of the m -th day, that is the maximum total number of position where more than one soldier resides.

Reasoning The monotonic condition is obvious, if i can occupy n bases in m days then i can also occupy $n - 1$ bases in m days. How to determine the maximum number of days needed to occupy v bases. Observe that the bases to occupy to have v bases in the least amount of time have to be connected. Note that to create a disconnected base configuration we have to occupy a base i and then remove all the soldiers from that base, this of course can't lead to the maximum. Now analyze the case where i can only move in one direction, it is always optimal to accumulate elements in the base k until $v - 1$ troops appeared, then carry all the soldiers to occupy $v - 1$ positions. It is optimal to make the interval as symmetric as possible with respect to k , and to fill the smaller side first and then the larger side. Accumulating is the optimal strategy since to fortify u bases on some given direction next to some base B we need $u + 1$ soldiers there, so there are two options (have already u soldier and wait for the $u + 1$ element, this takes $|k - B|$ time, this is minimized for $B = k$ that gives off the optimal strategy. Given that when two direction are available, any direction should be completed in one (time-consecutive sweep) then the optimal strategy always involves to fill the smaller side first, since filling the greater side would generate more soldier then needed in k , while sweeping the first direction.

```

int n, m, k;
void solve(){
    cin >> n >> m >> k;
    function<bool(int v)> can = [&](int v){
        if(v > n)
            return false;
        int tmp = v - 1;
        int l_s, r_s;
        bool lbr = (k - 1 <= n - k);
        if(lbr){
            l_s = min(k - 1, tmp / 2);
            r_s = tmp - l_s;
        } else {
            r_s = min(n - k, tmp / 2);
            l_s = tmp - r_s;
        }
        int mini = min(l_s, r_s), maxi = max(l_s, r_s);
        int diff = maxi - mini;
        return (maxi - mini + (mini - 1) + mini + maxi <= m) ;
    };
    int l = 1, r = n, ans = 2;
    while(l <= r){
        int mid = l + (r - 1) / 2;
        if(can(mid)){
            ans = mid;
            l = mid + 1;
        } else {
            r = mid - 1;
        }
    }
    cout << ans << '\n';
}

```

Complexity $O(\log(n))$

Rae Taylor and Trees

Source: Codeforces round 1065 (1600)

Link: <https://codeforces.com/contest/2171/problem/D>

Problem Statement Given a permutation p^* of n elements. Determine if it's possible to construct a tree of vertices $1, \dots, n$ out of the permutation, given that i can connect two vertices $1 \leq u < v \leq n$ if and only if u comes before v in the permutation. p^* .

Reasoning A tree is just a connected graph, without cycles, cycles are impossible to create in any case since given cycles $cy = \{(u, v)(v, w)(w, u)\}$ this means that u comes before v , then v comes before w and w comes before u which is a contradiction, so any graph that is constructed on the aforementioned rules, is a tree or a forest of trees. The key takeaway is that we need just to check connectedness, therefore any vertex should reach every other vertex.

Connectedness Notice that in case n was at the first position then no other element can connect to it, a mirrored argument would be constructed for 1 at the n th position.

We can extend the argument in this way: given $P[i] = \min(p^*[1], \dots, p^*[i])$ and the $S[i] = \max(p^*[i], \dots, p^*[n])$ and suppose we start constructing the tree with the first element being the current root, (the prefix tree would be the main component and the root is always the minimum element), our goal is to connect every node to the prefix

tree. Then $\forall i > 1$ if $p^*[i] > P[i-1]$ i can connect it to some element in the main component, if $p^*[i] > P[i-1]$ then it can be connected to the root of the prefix tree, $p^*[i] = nr$ would have to be the new root.

Rerooting To connect the new root to the old root, we need to find an element $p^*[j] > nr$ with $j > i$, and it is necessary that $p^*[j] > P[i-1] = or$, to do this take the maximum $max = p^*[j]$ such that $j > i$, if the maximum can't satisfy $max > P[i-1]$ then the tree is not constructible. So conversely the tree is constructible then the maximum can always be an articulation point between the new and the old root. Note that we can precalculate the roots, and the articulation points with $P[i-1]$ and $S[i+1]$.

```
int n, p[nxm], pref[nxm], suf[nxm];
void solve(){
    vec<pii> tree;
    cin >> n;
    rep(i, 1, n)
        cin >> p[i];
    pref[0] = n + 1;
    rep(i, 1, n)
        pref[i] = min(pref[i - 1], p[i]);
    suf[n + 1] = 0;
    rrep(i, n, 1)
        suf[i] = max(suf[i + 1], p[i]);
    bool nc = false;
    rep(i, 2, n){
        if(pref[i - 1] < p[i]){
            tree.pb({pref[i - 1], p[i]});
        } else {
            if(pref[i - 1] > suf[i + 1])
                nc = true;
            tree.pb({p[i], suf[i + 1]});
            tree.pb({pref[i - 1], suf[i + 1]});
        }
    }
    if(nc){
        cout << "No\n";
        return;
    }
    sort(all(tree));
    tree.erase(unique(all(tree)), e(tree));
    cout << "Yes\n";
    rep(i, 0, sz(tree) - 1)
        cout << tree[i].F << ' ' << tree[i].S << '\n';
}
```

Division versus Addition

Source: Squarepoint challenge (1700)

Link: <https://codeforces.com/contest/2152/problem/D>

Problem Statement Alice and Bob today have decided to play a game with their brand new array! Given an array a such that $\forall i \in [1, m] \quad a_i \geq 1$, Alice starts the game by choosing an index i where $1 \leq i \leq m \quad a_i \geq 2$ and replaces a_i with $\lfloor \frac{a_i}{2} \rfloor$, the next turn Bob plays, he wants to prove that the addition is strong compared to the division (the beloved operation of Alice) by that he chooses $1 \leq i \leq m \quad a_i \geq 2$ and replaces a_i with $a_i = a_i + 1$. They keep playing until every element is 1. Given an array the brand new array a and m queries, where each query is denoted by l and r , determine for each query q_i what's the minimum number of moves Alice has to play to finish the game on the array $a_l, \dots, a_r$ with optimal play by both sides.

Reasoning Observe that Alice has to make at least

$$\sum_{i=l}^r \log_2 a_i$$

Observe the following cases 4, 5, 6. If 6 is divided, then Bob can increase $a_i/2$ yielding one more move, 5 can be increased to return in a similar situation, we need to observe that by optimal play Alice at each turn will try to divide at least 5 or 3 one of them, effectively this yields $(\sum_{i=l}^r [a_i = 3 \vee a_i = 5])/2$ additional turns for Alice. Observe the following reasoning to generalize for classes of numbers:

classes of number A number yields a point on increment if by adding one i can make the situation $n+1 = 2^{k+1}$ where $\log_2 n = k$. Take an array composed only of 5, i can observe that i, Alice, have to divide all the 5 that i can so that they won't transform in 6, unfortunately i can divide only half of the fives. More generally if a number can be reduced to 0b101 it is a forceful number f_n . Observe that any number that can be reduced to 0b110 is indeed a guaranteed point r_n . Bitwisely, observe that 0b10...10 can be reduced in this way $0b10...1 \rightarrow 0b10...10 \rightarrow \dots \rightarrow 0b1010 \rightarrow 0b101 \rightarrow 0b110 \rightarrow 0b11 \rightarrow 0b100$ and by that any $2^k + 2 \leq n < 2^{k+1}$ is reduceable. Similarly we can derive that $2^k + 1$ are forceful. Finally, half of the remaining $y = 2^k$ numbers can become forceful, this is if there are no forceful number to increment, or no numbers to keep forceful, but if there are no forceful numbers at each turn i can create at max one forceful options that gets divided immediately by Alice yielding no points in the end. Imagine that if we make an element forceful then another element will become not forceful.

edge case Observe that 3 is a forceful number.

```
int n, q, a[250001], pos[250001]{}, nte[250001]{}, te[250001]{};
void solve(){
    cin >> n >> q;
    rep(i, 1, n)
        cin >> a[i];
    rep(i, 1, n){
        rrep(j, 31, 0)
            if(a[i] & (1 << j)){
                pos[i] = pos[i - 1] + j;
                break;
            }
    }
    rep(i, 1, n){
        int curr_p = pos[i] - pos[i - 1];
        bool nf = false, f = false;
        if(curr_p > 1){
            int mskf = (1 << curr_p) | 1;
            int msknf = (1 << curr_p);
            nf = (mskf & a[i] == mskf);
            if(!nf)
                f = ((msknf | a[i]) != msknf);
        }
        if(!nf)
            nf = (a[i] == 3);
        te[i] = te[i - 1] + f;
        nte[i] = nte[i - 1] + nf;
    }
    rep(i, 1, q){
        int l, r;
        cin >> l >> r;
        cout << (pos[r] - pos[l - 1]) + (nte[r] - nte[l - 1]) / 2 + te[r] - te[l - 1] << '\n';
    }
}
```

Difficulty: got all the observation right away, forceful and reduceable numbers, didn't generalize for 5 and 6 classes. Advised step (notice that at max 1 more point per number) notice the last tc and try manually to find a way to get a point out of every element

Unfair Game

Source: Codeforces round 1072 (1600)

Link: <https://codeforces.com/problemset/problem/2184/D>

Problem Statement Alice and Bob wants to play a game, Bob thinks of a number (initially) Alice knows if the number is odd or even. At each turn Alice can decide to subtract one or to halve the number. If the number is odd then alice can only subtract one. At the end of each turn Bob will tell Alice what is the biggest power of two that divides n . If Alice reaches zero then Bob will reply with -1 and ALice wins. Bob always wants to win this game though, so it gives Alice at max k moves. Find all the numbers a , $1 \leq a \leq n$ that Bob can think about that forces a loss for Alice.

Reasoning Observe that the best thing for Alice to do is divide the number by two if the number is even, if it's odd she is forced to subtract one. Suppose Alice begins the round with a number n whose binary representation ends with a zero (n is even). Notice that the division can be seen as a subtraction of $(n/2)$ then $\frac{n}{2} \geq 1$ for all $n \geq 1$. So dividing by two when i can is always better than subtracting one. Now what we can do to make Alice spend more moves is to increase the number of division and to increase the number of odd numbers that we'll get though division. Given n then the minimum number of moves to reach zero from n is $\log_2 n + cnt_o$ where cnt_o represents the number of ones in the binary representation of n . We'll use dynamic programming to count all the numbers that satisfy this property inside the interval $[1, n]$. We'll define $dp[d][tight][lead][n_o]$ as groups of numbers that are invariant for the aforementioned property, this reduces the search space to from n to $2(\log_2 n)^3$ which is good enough for the constraints. Tight is useful to not explore numbers that are bigger than n in the recursion tree. $lead$ and n_o uniquely defines a group of numbers so are needed in the memoization, and the position on which we are iterating (d) tells us how many bit we can flip. Even if the raw implementation is good enough, cuts of the recursion tree can be made (not explore if there's not enough ones to flip to reach condition). What if the interval is always $[1, 2^x]$, does this simplify the algorithm, what would be avoided ?

```
const int lnxm = 30;

int n, k;
int dp[lnxm][2][lnxm][lnxm];
int dfs(int d, bool tight, int lead, int n_o){
    if(d < 0)
        return (n_o ? n_o + lead > k : 0);
    if(dp[d][tight][lead][n_o] != -1)
        return dp[d][tight][lead][n_o];
    int cnt = 0;
    int dig = (bool)((1 << d) & n);
    for(int i = 0; i <= 1; i++){
        if(tight && i > dig)
            break;
        int nl = lead;
        if(lead == -1)
            nl = (i == 0 ? -1 : d);
        cnt += dfs(d - 1, tight && (i == dig), nl, n_o + i);
    }
    return dp[d][tight][lead][n_o] = cnt;
}

void solve(){
    cin >> n >> k;
    int pow = -1;
    for(int i = 31; i >= 0; i--){
```

```

    if((1 << i) & n){
        pow = i;
        break;
    }
}
memset(dp, -1, sizeof(dp));
cout << dfs(pow, true, -1, 0) << '\n';
}

```

Complexity Time: $O(\log_2^3 n)$ Space: $O(\log_2^3 n)$

Cluster Computing System

Source: 2025 ICPC Asia Taichung Regional Contest (1500)

Link: <https://codeforces.com/contest/2172/problem/F>

Problem Statement Given n servers each having a protocol with period p_i with $1 \leq i \leq n$. Define a connection between two server u, v s.t. $1 \leq u < v \leq n$. The cost of installing a connection between two server is $\gcd(p_u, \dots, p_v)$. We would also like all servers to be connected in some way (More formally From any server u we can reach any server v through path composed of 1 or more connections). Print the minimum possible cost to achieve such configuration.

Reasoning Let's look at a random pair of servers u, v where $u < v$. The cost of connecting these two servers is $c_{uv} = \gcd(p_u, \dots, p_v)$, observe that if we wanted to connect v with a server $u' < u$ then the cost c' would be $c_{u'v} = \gcd(p'_{u'}, \dots, p_{u-1}, c_{uv})$. Remember that mathematically the $\gcd$ is $\gcd(u, v) = \max(\bigcap_{i=u}^n D_{p_i})$, the more elements are added the less the intersection is gonna be, the less the max is gonna be. So $c_{u'v} \leq c_{uv}$. Once we proved this is trivial to deduce the minimum cost connection has to be $(1, n)$ since this has to have the minimal $\gcd$ out of all pairs. For this network to be connected any server should be connected note, so there should be at least an edge $\exists(i, j) \forall i \in [1, N]$. Note that the two possible optimal ways to connect i to some other node j could be $(i, 1)$ or (i, n) , since $c(i, 1) \leq c(i, j)$ s.t. $1 < j < i$ and $c(i, n) \leq c(i, j)$ s.t. $i < j < n$. For each i the cost of connecting i to some other server is $c(i) = \max(c(1, i), c(i, n))$. The optimal construction is to connect all server i where $1 < i < n$ to either 1 or to n . Then build a bridge between 1 and n with cost $\gcd(u_1, \dots, u_n)$. This is indeed the best possible construction since it includes the minimum cost connection, for each server i is established the minimum possible connection cost to connect i to the cluster, and it's composed of $n - 1$ connections (which is the minimum number of connection needed).

```

public class Main {
    // header
    public static int gcd(int a, int b) {
        int tmp;
        while(b != 0) {
            tmp = b;
            b = a % b;
            a = tmp;
        }
        return a;
    }
    public static void main(String[] args) throws IOException {
        FastRead in = new FastRead();
        BufferedWriter out = new BufferedWriter(new OutputStreamWriter(System.out));
        int N = in.nextInt();
        int[] P = new int[N + ArrPadd];
        int[] Suf = new int[N + ArrPadd];
        int[] Pref = new int[N + ArrPadd];
        for(int i = 1; i <= N; i += 1) {
            P[i] = in.nextInt();

```

```

    }
    Pref[1] = P[1];
    for(int i = 2; i <= N; i += 1) {
        Pref[i] = gcd(Pref[i - 1], P[i]);
    }
    Suf[N] = P[N];
    for(int i = N - 1; i >= 1; i -= 1) {
        Suf[i] = gcd(Suf[i + 1], P[i]);
    }
    long cost = Pref[N];
    for(int i = 2; i <= N - 1; i += 1) {
        cost += (Suf[i] < Pref[i] ? Suf[i] : Pref[i]);
    }
    out.write(Long.toString(cost));
    out.newLine();
    out.flush();
}
}

```

Complexity $O(n \log_2 n)$

Sub-RBS

Source: Codeforces Round 1073 (1400)

Link: <https://codeforces.com/contest/2190/problem/B1>

Problem Statement Given two strings a and b composed only by (or), a is better than b if at least one of the following condition is true:

- b is a prefix of a with $a \neq b$
- Denote as i the first index such that $a_i \neq b_i$ then $a_i = ($ and $b_i =)$.

Define a string, composed only of $[(,)]$, as regular if and only if there's no way of adding operators and operations in the string that results in a non sound mathematical expression. Given s , find the length of the longest regular subsequence $t^\dagger$ of s .

Reasoning First observe that $t^\dagger \neq s$ else no condition would be satisfied. Notice that for $t^\dagger$ to be a subsequence of s then $\text{len}(t^\dagger) \leq \text{len}(s)$, hence the first condition will never be true since a prefix is by definition shorter this would imply $\text{len}(s) < \text{len}(t^\dagger)$ which is contradictory. Once fixed i the second condition instead states that $t^\dagger(1, i - 1) = s(1, i - 1)$, this means that the subsequence and the string should be identical for the first $i - 1$ elements. More specifically we are comparing the first $i - 1$ elements of s with the first $i - 1$ elements of $t^\dagger$ which maps to $i - 1$ elements of s . Now observe that this map can be arbitrary as long as the prefixes are identical. Note also that $t^\dagger$ should be regular so in the end the number of open parenthesis (cnt_o) should be equal to the number of closed parenthesis (cnt_c). The final key observation is that any prefix has a number of parentheses to close that is ≥ 0 , by using the identity mapping for the prefix until the fixed index i , then we have more freedom to pick more closing braces in $t^\dagger$ to close the unclosed braces in the prefixes and to pair all the possible remaining closing braces. Given the two indices $j > i$ where $s_j = t_i^\dagger = (\neq s_i =)$, then choosing j as small as possible is optimal. Once j is fixed observe if there not exists j with such property or if the number of unclosed parenthesis in the prefix $+ 1$ (take into consideration s_j) is strictly greater than the number of closing braces in the suffix $[s_j, \dots, s_n]$ then it's insolvable for the prefix i else calculate the length as

$$\#_((P[i - 1] + i + 1 + 2 \cdot (\#p_{(h,k)}))$$

where $\#_(($ is the number of unclosed parentheses in the prefix, and $\#p(h, k)$ are all the free pairs $s_h = ($ and $s_k =)$ where $j < h, k \leq n$. Iterate though all prefixes and maximize.

```

public class Main {
    // header
    public static void main(String[] args) throws IOException {
        FastRead in = new FastRead();
        BufferedWriter out = new BufferedWriter(new OutputStreamWriter(System.out));
        int tc = in.nextInt();
        while(tc-- > 0) {
            int n = in.nextInt();
            String seq = in.nextLine();
            int maxLength = -1;
            int[] PrefBalance = new int[n + ArrPadd];
            int[] SufBalance = new int[n + ArrPadd];
            PrefBalance[0] = 1;
            for(int i = 1; i < n; i += 1) {
                PrefBalance[i] += PrefBalance[i - 1] + (seq.charAt(i) == '(' ? 1 : -1);
            }
            SufBalance[n - 1] = 1;
            for(int i = n - 2; i >= 0; i -= 1){
                SufBalance[i] += SufBalance[i + 1] + (seq.charAt(i) == ')' ? 1 : 0);
            }
            int rightClosing = -1, leftOpening = -1;
            for(int i = n - 1; i >= 0; i -= 1){
                if(seq.charAt(i) == '('){
                    rightClosing = SufBalance[i];
                    leftOpening = n - i - rightClosing;
                    continue;
                }
                if(PrefBalance[i - 1] + 1 <= rightClosing){
                    int rem = rightClosing - (PrefBalance[i - 1] + 1);
                    maxLength = Math.max(maxLength, i + PrefBalance[i - 1] + 2 + Math.min(rem,
                        leftOpening - 1) * 2);
                }
            }
            out.write(Integer.toString(maxLength));
            out.newLine();
        }
        out.flush();
    }
}

```

Complexity: $O(n)$

BattleCows

Source: Codeforces Round 1074 (1700)

Link: <https://codeforces.com/contest/2185/problem/F>

Problem Statement Farmer John signed up to compete for a cow tournament, the catch is that he can only bring one cow with him. Therefore John decides to organize q tournaments to decide which cow of the 2^n present has to bring. At the beginning of the tournament every cow i is placed in the slot i and has a value c_i where $1 \leq i \leq 2^n$, then at each round any slot i odd challenges the adjacent slot to the right $i + 1$. Define the value of a slot i as $\bigoplus_j c_j$, the xor of all cows values c_j present in the slot i . The slot that has bigger value wins (if the value is equal the leftmost wins), so all cows in the winning slot goes on top of all the cows in the losing slot, the winning slot is therefore taken out since it was emptied the round before. Repeat until only one slot remains. John wants to know how many cows will the cow c_i have on top of it in the final slot. Moreover John at any tournament test a potion that changes the value of the cow to be analyzed.

Reasoning Notice that all the challenges in which the cow c_i participated is fixed. By adding a potion the order in the slot will be different but the cows i will be the same. For example, given an array of slots $a = a_i, \dots, a_{2^n}$, in the semifinals the two slots that will challenge each other will always be $s^* = \{1, \dots, 2^{n-1}\}$ and $s^\dagger = \{2^{n-1} + 1, \dots, 2^n\}$. The values of the cows will change the order of cows in such slots. Note that, by going top down, we know in which slot the analyzed cow is in. For example if our cow was positioned in the first slot then it will appear in s^* wlog let's assume our cows is in s^* . If $\bigoplus s^* > \bigoplus s^\dagger \vee (\bigoplus s^* = \bigoplus s^\dagger \wedge s^* \text{ is odd})$ we know for sure that 0 cows of the opposing slot $s^\dagger$ will be on top else 2^{n-1} of the opposing slot will be on top. Now recursively analyze s_1^* and $s_1^\dagger$ which challenges each other to form the previously analyzed s^* . To calculate $\bigoplus s^*$ and $\bigoplus s^\dagger$ we can use a xor segment tree. This data structure perfectly simulates the tournament FBT structure, and can store all the intermediate

$$\left(\bigoplus s^*, \bigoplus s^\dagger \right)_1, \dots, \left(\bigoplus s^*, \bigoplus s^\dagger \right)_n$$

We can handle the potion modification swiftly, since the segment tree can handle a point update in $\log_2(n)$. The queries should be independents therefore we will have to reset the segment tree in between queries.

```
public class Main {
    // header
    static class SegTree {
        int[] st;
        int X;
        SegTree(int size, int[] arr){
            X = size;
            st = new int[X << 1];
            for(int i = 0; i < X; i += 1) {
                st[i + X] = arr[i];
            }
            for(int i = X - 1; i > 0; i -= 1) {
                st[i] = st[i << 1] ^ st[i << 1 | 1];
            }
        }
        void update(int i, int v) {
            for(st[i += X] = v; i > 1; i >>= 1) {
                st[i >> 1] = st[i] ^ st[i ^ 1];
            }
        }

        private int find(int i, int l, int r, int root) {
            int currCows = X >> 1, ans = 0;
            while(root != i + X) {
                int mid = l + (r - l) / 2;
                root <<= 1;
                l = (i > mid ? mid + 1 : l);
                r = (i <= mid ? mid - 1 : r);
                if(i > mid)
                    root |= 1;
                if(st[root] < st[root ^ 1])
                    ans += currCows;
                if(st[root] == st[root ^ 1] && i > mid) {
                    ans += currCows;
                }
                currCows >>= 1;
            }
            return ans;
        }

        int query(int i) {
            return find(i, 0, X - 1, 1);
        }
    }
}
```

```

}

public static void main(String[] args) throws IOException {
    FastRead in = new FastRead();
    BufferedWriter out = new BufferedWriter(new OutputStreamWriter(System.out));
    int tc = in.nextInt();
    while(tc-- > 0) {
        int n = in.nextInt();
        int q = in.nextInt();
        n = (1 << n);
        int[] s = new int[n];
        for(int i = 0; i < n; i += 1) {
            s[i] = in.nextInt();
        }
        SegTree seg = new SegTree(n, s);
        for(int i = 0; i < q; i += 1) {
            int b = in.nextInt();
            int c = in.nextInt();
            seg.update(b - 1, c);
            out.write(Integer.toString(seg.query(b - 1)));
            seg.update(b - 1, s[b - 1]);
            out.newLine();
        }
    }
    out.flush();
}
}

```

Complexity $O(qn)$

Optimization A purq is overkill, a binary search on a prefix xor is enough, notice that given that the value of any slot s^* containing the cow i (with original value c_i and new value p_j) is $V = (c_{2^{k+1}} \oplus \dots \oplus c_i \oplus \dots \oplus c_{2^{k+1}})$ so the updated value is $V^\dagger = V \oplus c_i \oplus p_j$. This works since all the previous point updates have gotten rolled back.

Product Queries

Source: Codeforces Round 1076 (1400)

Link: <https://codeforces.com/problemset/problem/2193/E>

Problem Statement Given an array a of size n , John would like to know the shortest sequence of picks $S = \{s_1, \dots, s_m\}$ $s_i \in a$ such that $\prod S = i$ for each i . Note that John can pick the same element a_i multiple times.

Reasoning Observe that this can be seen as an unbounded knapsack problem. Consider the following subproblem, $dp[j]$ shortest sequence S such that $\prod S = j$ then $j \in a \implies dp[j] = 1$ and $dp[j] = \min_i (dp[j/w[i]] + 1, dp[j])$. Observe that $w[i]$ has to divide j , moreover $dp[j/w[i]]$ has to be constructible. Since we don't really care about the position of the elements in the sequence we can invert the iteration logic without losing in generality. By switching the loops we are building sorted invariants of the same sequence. In this case we notice that j has to be a multiple of $w[i]$, so by fixing $w[i]$ we iterate on all multiples $kw[i]$. Finally we can erase all duplicates a_i since John can pick the same a_i multiple times.

```

void solve(){
    int n;
    cin >> n;
    vector<int> a(n), dp(n + 1, Sinf);
    for(int& ai : a)

```

```

    cin >> ai;
    sort(all(a));
    a.erase(unique(all(a)), end(a));
    for(int i = 0; i < sz(a); i++){
        dp[a[i]] = 1;
        for(int j = a[i] * 2; j <= n; j += a[i]){
            if(dp[j / a[i]] != Sinf){
                dp[j] = min(dp[j], dp[j / a[i]] + 1);
            }
        }
    }
    for(int i = 1; i <= n; i++){
        cout << (dp[i] == Sinf ? -1 : dp[i]) << " \n"[i == n];
    }
}

```

Complexity $O(k \log n)$

Rabbits

Source: Codeforces Global Round 29 (1500)

Link: <https://codeforces.com/problemset/problem/2147/C>

Problem Statement Julia has n pots represented by a binary string s . In some pots there are plants $s_i = 1$ while others are empty $s_i = 0$. She would like to put one rabbit in every empty pot. She can choose whether the rabbit faces left or right. A Rabbit always jump on the pot in front of them, if there's no rabbit already in the landing pot or there is no other rabbit that had chosen that landing pot as well. Julia would like to take a photo, to do so no rabbit should move. Determine if such configuration exists.

Reasoning Notice that any island $S(l, r)$ of empty pots with $l < r$ can have a rabbits assignment that makes them standing still. To do so the rabbit in the rightmost pot should face left, while the rabbit in the leftmost pot should face right. Let's now call define dangerous island (an empty pot surrounded with planted pots, more formally these are islands of length 1). For each dangerous island i there should be at least another rabbit on the other side of the plant to the left or to the right else there's no valid configuration since the rabbit i is allowed to jump to the plant $i + 1$ or to the plant $i - 1$. On the other side there can be another dangerous island or a not dangerous one. Now observe the following pot configuration $s = PRPRPRP$, notice that the rabbit in the middle pot is overloaded. Analyze all chains in the binary string, a chain produces overloading if and only if all the chained rabbits are forceful and the number of overloaded rabbits is odd. For the first part observe that if a rabbit at the beginning or at the end of the chain is part of a not dangerous island then it doesn't need to be part of the chain to not jump (it can look to his neighbor), more specifically if the number of chained rabbit is odd it will participate otherwise it will not. Any rabbit that can choose to not stay in the chain to not jump will be denoted as not forceful. If a rabbit is staying in the pot 1 or in the pot n it will never be forceful as he can choose to not jump freely, any other rabbits part of a dangerous island is forceful.

```

int n;
string s;
bool fcf1[nxm]{};
void solve(){
    cin >> n >> s;
    bool ok = true;
    memset(fcf1, 0, sizeof(fcf1));
    for(int i = 1; i < n - 1; i++){
        if(s[i] == '0')
            if(s[i + 1] == '1' && s[i - 1] == '1'){
                if((i - 2 < 0 || s[i - 2] == '1') && (i + 2 >= n || s[i + 2] == '1'))
                    ok = false;
            }
    }
}

```

```

        fcfl[i] = true;
    }
    if(!ok){
        cout << "no\n";
        return;
    }
    int pc = s[0], cnt_r = (s[0] == '0'), cnt_fcfl = 0;
    for(int i = 1; i < n; i++){
        if(s[i] == pc){
            if(cnt_r > 2 && (cnt_fcfl == cnt_r) && (cnt_r & 1))
                ok = false;
            cnt_fcfl = 0;
            cnt_r = 0;
        }
        cnt_r += (s[i] == '0');
        cnt_fcfl += fcfl[i];
        pc = s[i];
    }
    if(cnt_r > 2 && (cnt_fcfl == cnt_r) && (cnt_r & 1))
        ok = false;

    cout << (ok ? "yes\n" : "no\n");
}

```

Meximum Array

Source: Codeforces Round 1066 (1400)

Link: <https://codeforces.com/contest/2157/problem/C>

Problem Statement Given 3 positive integers n, k, q , then construct an array such that for each query $q = (c, l, r)$, if $c = 2$ then $\text{MEX}(a_l, \dots, a_r) = k$ while if $c = 1$ or $\min(a_l, \dots, a_r) = k$.

Reasoning Notice that for each $q_i = (1, l, r)$ then for all $l \leq i \leq r$ is valid $a_i \geq k$. For each $q_i = (2, l, r)$ then $r - l + 1 \geq k$ there should not be any $a_i = k$. If an index is only part of a mex query then $a_i < k$, if is only part of a min query $a_i = k$, if it is part of both $a_i > k$. Think now of the sequence $S = \{i, j, \dots, k\}$ where every s is part of only mex queries. For the array to be constructible any MEX Query will query at least k adjacent spots in the sequence, moreover only the spots that are in the sequence are functional for $\text{MEX}(l, r)$ to be equal to k . To distribute nicely the $v \in 0, k - 1$ observe that any window (subarray) w of size k of S should have $\text{MEX}(w) = k$. Any shifted modular progression $0, 1, 2, \dots, k - 1, 0, 1, \dots$ works and like that any reversed shifted progression works $k - 2, \dots, 1, 0, k - 1, k - 2$.

```

void solve(){
    int n, k, q;
    cin >> n >> k >> q;
    int arr[n + 1];
    for(int i = 1; i <= n; i++){
        arr[i] = k;
    }
    bool mexed[n + 1];
    bool mined[n + 1];
    memset(mined, 0, sizeof(mined));
    memset(mexed, 0, sizeof(mexed));
    for(int i = 0; i < q; i++){
        int l, r, c;
        cin >> c >> l >> r;
    }
}

```

```

    for(int j = 1; j <= r; j++){
        if(c == 1){
            mined[j] = true;
            continue;
        }
        mexed[j] = true;
    }
}
int cnt = 0;
int dir = 1;
for(int j = 1; j <= n; j++){
    if(mexed[j] && !mined[j]){
        arr[j] = cnt;
        cnt += dir;
        if(cnt == k){
            cnt = 0;
        }
    }
    if(mexed[j] && mined[j]){
        arr[j] = k + 1;
    }
}
for(int i = 1; i <= n; i++){
    cout << arr[i] << ' ';
}
cout << '\n';
}

```

Christmas Tree Decorations

Source: Educational Codeforces Round 186 (1600)

Link: <https://codeforces.com/contest/2182/problem/D>

Problem Statement Given n persons that wants to decorate the tree and $n + 1$ boxes of decoration, at each turn i the person p_i has to pick on decoration from the box b_0 or b_i . The turns follow a permutation $p^\dagger$. A permutation is considered valid if at no point in the game (unless it's the last turn) a person has no decorations to pick. Count the number of all possible valid permutations.

Reasoning Notice that for the game to finish there should no more decorations in any box. At each round (a full cycle of $p^\dagger$) each player takes one decoration from their own box or the shared box. Taking from the shared box before emptying the personal box is considered not optimal, since we would like to analyze the permutation under optimal play we'll discard this possibility. Imagine all players taking from the shared box, eventually the one with less decorations in the personal box won't be able to pick any decorations with the game being still ongoing. Once every player has finished his own decorations the game goes on without problems, and any ordering is correct from that point on. Define $S \subseteq \{1, \dots, n\}$ as the persons with the largest amount of personal decorations M . Let's see what happens after $M - 1$ turns: First notice that $d_0 = d_0 - \sum_{i=1}^n \max(M - 1 - d_i, 0)$, which represent the leftovers in the shared box before the deciding turn. All the players that don't belong to S can't take before a player in S when there's no decorations in the shared box. By that we can solve the problem using Dynamic Programming. Define $dp[d_0][n_s][n_o]$ where n_s are the number of players who still have one personal decoration (notice that if a player with no personal decoration can't do anything while another has still personal decoration the permutation is invalid), n_o the number of players who have to pick from the shared box and d_0 the number of decoration in the shared box. Define the transition:

$$dp[d_0][n_s][n_o] = n_s * dp[d_0][n_s - 1][n_o] + [n_o > 0 \wedge d_0 > 0] * n_o * dp[d_0 - 1][n_s][n_o - 1]$$

And the base case:

$$dp[d_o][0][n_o] = n_o!$$

Edge case if $d_0 < 0$ then there's no way to end the game before such a situation happens. If $d_0 \geq n_o$ then all the empty handed players can take something from the shared box, hence no conflicts so all permutations are valid ($n!$).

```
const int dim = 51;
int n, a[dim];
ll dp[dim][dim][dim], f[dim];

ll dfs(int sh, int fin, int oth){
    if(fin == 0)
        return f[oth];
    if(dp[sh][fin][oth] != -1)
        return dp[sh][fin][oth];
    ll ans = (1ll * fin * dfs(sh, fin - 1, oth)) % mod;
    if(sh > 0 && oth > 0)
        ans = (1ll * oth * dfs(sh - 1, fin, oth - 1) % mod + ans) % mod;
    return dp[sh][fin][oth] = ans;
}

void solve(){
    cin >> n;
    for(int i = 0; i <= n; i++)
        cin >> a[i];
    int mx = *max_element(a + 1, a + n + 1), cnt = 0;
    for(int i = 1; i <= n; i++)
        a[0] -= max(0, mx - 1 - a[i]);
    for(int i = 1; i <= n; i++)
        cnt += (a[i] == mx);
    if(a[0] < 0){
        cout << "0\n";
        return;
    }
    if(a[0] >= n - cnt){
        cout << f[n] << '\n';
        return;
    }
    memset(dp, -1, sizeof(dp));
    cout << dfs(a[0], cnt, n - cnt) << '\n';
}

void pre(){
    f[0] = 1; f[1] = 1;
    for(ll i = 2; i < dim; i++)
        f[i] = (f[i - 1] * i) % mod;
}
```

Restricted sorting

Source: Codeforces Round 1077 (1400)

Link: <https://codeforces.com/contest/2188/problem/C>

Problem Statement Given an array a we would like to sort it in a non descending order. Given a fixed integer k i can swap two elements a_i, a_j if and only if $|a_i - a_j| \geq k$. Find the largest k such that the array can be sorted given that we can do any amount of swaps operations as long as they follow the aforementioned rule.

Reasoning Notice that given two elements in the wrong position i, j we can swap them if and only if $|a_i - a_j| \geq k$ if not we need to find another element, it's provable that we need just one element to do the swap, for any pair a_i, a_j the optimal choice would always be to take the min mn or the max mx as a pivot. Given mn, i, j, mx we can do the following $(mn, mx, j, i) \rightarrow (j, mx, mn, i) \rightarrow (j, mx, mn, i) \rightarrow (mx, j, i, mn) \rightarrow (mn, j, i, mx)$, So the cost to put j in the right position is $mx - a_j$ while the cost to put i in the right position is $a_j - mn$. Without binary search just maximize the function on a specific bound, for each element. By this we can move any element wherever we want.

```
int n, a[nxm], srt[nxm];
void solve(){
    vector<int> ch;
    cin >> n;
    for(int i = 0; i < n; i++)
        cin >> a[i];
    memcpy(srt, a, sizeof(a));
    sort(srt, srt + n);
    for(int i = 0; i < n; i++){
        if(srt[i] != a[i])
            ch.pb(srt[i]);
    }
    if(!sz(ch)){
        cout << "-1\n";
        return;
    }
    int ans = INT_MAX, mini = srt[0], maxi = srt[n - 1];
    for(int& a : ch)
        ans = min(max(a - mini, maxi - a), ans);

    cout << ans << '\n';
}
```

The ancient wizards capes

Source: Codeforces Round 1056 (1500)

Link: <https://codeforces.com/contest/2155/problem/C>

Problem Statement Given n wizards $w_1, \dots, w_n$ each wizard can yield an invisibility cape, and can face left or right. John at each point i counts how many wizards can he see. John can always see the wizard at position i , no matter the orientation. Given the John's record, count how many possible configurations of wizards could have been there.

Reasoning Notice that between r_i and r_{i+1} the number of visible can stay the same, increase by one or decrease by one. Let's think about john sweeping from left to right. If $r_{i+1} > r_i$ w_i has to face left, and the same goes for w_{i+1} . A mirrored argument is made for $r_{i+1} < r_i$. Finally if $r_{i+1} = r_i$ then they have to be face inwards or outwards. Notice that if we know that orientation of two wizards then all the other wizards will follow, by that there will be at most 2 configurations when $(\#)\forall_i r_{i+1} \neq r_i$, else it's guaranteed that the one inequality infers all the other orientation. The possible () configurations are $<><>$ and $><><$, $<><><$, $><><>$. For even configurations then the valid r_i values are $n/2 + 1$ or $n/2$, and each value is mapped to only one configuration. For odd configurations the valid value is just $n/2 + 1$ and maps to exactly two different configurations. In the case exists at least one i such that $r_i \neq r_{i+1}$ then just build manually the string by inferring on the $r_i = r_{i+1}$ pairs. Then compute the partial results like it's described in the problem statement via prefix and suffix array and check that the results are consistent else 0 configuration found.

```
enum dir{
    leftt,
    rightt,
```

```

};
int next(int prev){
    return (prev == leftt ? rightt : leftt);
}
void solve(){
    int n;
    cin >> n;
    vector<int> a(n), st(n, -1);

    for(int& e : a)
        cin >> e;
    int j = -1;
    for(int i = 1; i < n; i++){
        if(abs(a[i] - a[i - 1]) >= 2){
            cout << 0 << '\n';
            return;
        }

        if(a[i] != a[i - 1]){
            st[i] = (a[i] < a[i - 1] ? rightt : leftt);
            if(st[i - 1] != -1 && st[i] != st[i - 1]){
                cout << 0 << '\n';
                return;
            }
            st[i - 1] = st[i];
            j = i;
        }
    }
    if(j == -1){
        cout << (n & 1 ? (a[0] != (n / 2 + 1) ? 0 : 2)
            : (a[0] < (n / 2) || a[0] > ((n / 2) + 1) ? 0 : 1)) << '\n';
        return;
    }

    int prev = st[j - 1];
    for(int i = j; i < n; i++){
        if(st[i] == -1){
            prev = next(prev);
            st[i] = prev;
        }
        prev = st[i];
    }

    prev = st[j];
    for(int i = j - 1; i >= 0; i--){
        if(st[i] == -1){
            prev = next(prev);
            st[i] = prev;
        }
        prev = st[i];
    }
    vector<int> pref(n), suf(n);
    pref[0] = (st[0] == leftt);
    for(int i = 1; i < n; i++)
        pref[i] = pref[i - 1] + (st[i] == leftt);
    suf[n - 1] = (st[n - 1] == rightt);
    for(int i = n - 2; i >= 0; i--)

```

```

    suf[i] = suf[i + 1] + (st[i] == rightt);
for(int i = 0; i < n; i++){
    int l = (!i ? 0 : pref[i - 1]);
    int r = (i == n - 1 ? 0 : suf[i + 1]);
    if(a[i] != l + r + 1){
        cout << 0 << '\n';
        return;
    }
}
cout << 1 << '\n';
}

```

Fragmented Nim

Source: 2025-2026 ICPC, NERC, Northern Eurasia Finals (1600)

Link: <https://codeforces.com/contest/2181/problem/F>

Problem Statement Alice and Bob are playing a game: given some piles of stones $p^\dagger = p_1, \dots, p_n$, at each turn the opponent player chooses a non empty pile i s.t. $1 \leq i \leq n \wedge p_i > 0$ then the current player can choose to take any number of stones from such pile p_i . Who takes the last stone wins the game. Given n and $p^\dagger$ determine who wins with optimal play by both sides, Alice starts the game.

Reasoning Let's start with the trivial case first, if there's only one pile, bob is forced to choose that pile and alice is gonna finish it, so alice is winning. Let's analyze the game with more piles, take this configuration as an example $p^\dagger = \{p_1, \dots, p_n\}$, Alice in this case can force a win, observe that no matter what Bob chooses, Alice can take $p_i - 1$ stones from it and then indicate the very same pile p_i , Bob is forced to finish it, this interaction goes on until Bob chooses the last pile for Alice, at that point Alice wins by taking all stones from it. This strategy has a weakness: any pile made by only one stone can stop Alice from her winning sequence. Given a configuration with exactly one pile having only one stone, Bob has to choose it for Alice (Alice would be forced to take one stone and then the situation is identical of the previous case but with inverted roles). Notice that if Bob didn't chose it, Alice would have been in the winning position since she would assign it to Bob on the next turn. We know that if a one exists then a player is forced to assign it, to not lose. Imagine a configuration with m piles composed of one stones, then if $m \% 2 = 1$ Bob is winning else ($m \% 2 = 0$) Alice wins. Notice that if $m = n$ the situation is the opposite, since the would winning sequence start when the game has finished.

```

int n;
void solve(){
    cin >> n;
    vector<int> a(n);
    for(int& e : a)
        cin >> e;
    if(n == 1){
        cout << "Alice\n";
        return;
    }
    int cnt = 0;
    for(const int& e : a)
        cnt += (e == 1);

    if(cnt == n){
        cout << (!(n & 1) ? "Bob\n" : "Alice\n");
        return;
    }
    cout << (cnt & 1 ? "Bob\n" : "Alice\n");
}

```

Complexity $O(n)$

Mexification

Source: Codeforces Round 1047 (1500)

Link: <https://codeforces.com/contest/2137/problem/E>

Problem Statement Given an array $a^\dagger = a_1, \dots, a_n$, we execute k times the following operation:

$$a_i = \text{MEX}(a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$$

Return $\sum_i^n a_i$ after k operations.

Reasoning Observe that after 1 operation the mex could only $\text{MEX}(a_1^\dagger) < \text{MEX}(a_0^\dagger)$ or $\text{MEX}(a_1^\dagger) = 1 + \text{MEX}(a_0^\dagger)$ or $\text{MEX}(a_0^\dagger) = \text{MEX}(a_1^\dagger)$. The first case occurs when there is a duplicate $a_i < \text{MEX}(a_0^\dagger)$, while the second case occurs when there are no duplicates and there is at least one $a_i > \text{MEX}(a_0^\dagger)$ when no duplicates are found and there is no element a_i bigger than the mex of the array then the array will never change. Let's make an example of the first case

$$\begin{aligned} a_0^\dagger &= 0, 1, 1, 2, 2, 2, 3, 4, 6, 6 \\ a_1^\dagger &= 0, 5, 5, 5, 5, 5, 3, 4, 5, 5 \\ a_2^\dagger &= 0, 1, 1, 1, 1, 1, 1, 1, 1, 1 \\ a_3^\dagger &= 0, 2, 2, 2, 2, 2, 2, 2, 2, 2 \\ a_{2k}^\dagger &= a_2^\dagger \wedge a_{2k+1}^\dagger = a_3^\dagger \end{aligned}$$

Given $M_i = \text{MEX}(a_i^\dagger)$ We can reduce any configuration of the first type as

$$f_1(k) = \begin{cases} \sum_{i=0}^n a_i & \text{if } k = 1 \\ ((M_1 - 1) * M_1) / 2 + M_1 * (n - M_1) & \text{if } k \text{ is odd} \\ ((M_1 - 1) * M_1) / 2 + (M_1 + 1) * (n - M_1) & \text{if } k \text{ is even} \end{cases}$$

Let's look at two configuration of the second type:

$$\begin{aligned} a_0^\dagger &= 0, 1, 2, 3, 4, 5, 6, 7, 8, 10 & b_0^\dagger &= 0, 1, 2, 3, 4, 6, 6, 7, 8, 9 \\ a_1^\dagger &= 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 & b_1^\dagger &= 0, 1, 2, 3, 4, 5, 5, 5, 5, 5 \\ a_2^\dagger &= 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 & b_2^\dagger &= 0, 1, 2, 3, 4, 6, 6, 6, 6, 6 \\ a_3^\dagger &= 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 & b_3^\dagger &= 0, 1, 2, 3, 4, 5, 5, 5, 5, 5 \\ a_k^\dagger &= a_1^\dagger & b_{2n}^\dagger &= b_2^\dagger, \quad b_{2n+1}^\dagger = b_3^\dagger \end{aligned}$$

Given m the number of elements $a_i > \text{MEX}(a_0^\dagger)$, we can calculate the results for configuration on the second kind as

$$f_2(k) = \begin{cases} \sum_{i=0}^n a_i & \text{if } k = 1 \vee m = 1 \\ ((M_0 - 1) * M_0) / 2 + M_0 * (n - M_0) & \text{if } k \text{ is odd} \wedge m \neq 1 \\ ((M_0 - 1) * M_0) / 2 + (M_0 + 1) * (n - M_0) & \text{if } k \text{ is even} \wedge m \neq 1 \end{cases}$$

```
int n, k;
int calc_mex(vector<int>& a, vector<int>& u){
    fill(all(u), 0);
    for(const int& e : a){
        if(e > n)
            continue;
        u[e]++;
    }
    for(int i = 0; i <= n; i++){
```

```

        if(!u[i])
            return i;
    }
    return -1;
}

void solve(){
    cin >> n >> k;
    vector<int> a(n), u(n + 1, 0);
    for(int& e : a)
        cin >> e;
    int mex = calc_mex(a, u);
    ll sum = 0, cnt = 0;
    for(int& e : a){
        if(e > mex)
            e = mex;
        if(u[e] > 1)
            e = mex;
        sum += e;
    }
    if(!(k - 1)){
        cout << sum << '\n';
        return;
    }
    int prev = mex;
    mex = calc_mex(a, u);
    sum = 0;
    for(int &e : a){
        sum += (e < mex ? e : 0);
        cnt += (e > mex);
    }
    if(prev < mex){
        cout << (u[mex - 1] <= 1 ? sum : ((k - 1) & 1 ? sum + u[mex - 1] : sum)) << '\n';
        return;
    }
    cout << ((k - 1) & 1 ? cnt * mex + sum : cnt * (mex + 1) + sum) << '\n';
}

```

The Trail

Source: Codeforces Round 996 (1400)

Link: <https://codeforces.com/contest/2055/problem/C>

Problem Statement Given a grid G of n rows by m columns. We know that originally there was a magic number x such that for each j then $\sum_i^n g_{ij} = x$ and for each i then $\sum_j^m g_{ij} = x$. A path from p^* of length $n+m-1$ (from $(1,1)$ to (n,m)) of cells had been erased, find a way to fill the cells belonging to the path described by a string s where $s_i = \{D, R\}$ such that the aforementioned property is restored.

Reasoning Observe that for any grid that satisfies the property, it has to hold that

$$x * n = x * m \implies x(n - m) = 0$$

Note that by choosing $x = 0$ we are guaranteed that a configuration exists. Since x is now fixed we can calculate the first or last element of the path (we know that $(0,0)$ and $(n-1, m-1)$ are the only elements that are part of a row or of a column where no other erased cell lie). Wlog calculate $g(0,0)$, the next element can be calculated since now the same of $(0,0)$ and $(n-1, m-1)$ holds. Recursively set all the cells in the path until $(n-1, m-1)$ is reached. The last cell can be calculated from the row equation or the column equation (there's no difference since everything is now fixed).

```

int n, m;
ll g[mx1][mx1], c[mx1], r[mx1];
void solve(){
    string s;
    cin >> n >> m;
    cin >> s;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            cin >> g[i][j];
    memset(c, 0, sizeof(c));
    memset(r, 0, sizeof(r));
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            r[i] += g[i][j];
    for(int j = 0; j < m; j++)
        for(int i = 0; i < n; i++)
            c[j] += g[i][j];
    int h = 0, k = 0;
    for(int i = 0; i < sz(s); i++){
        g[h][k] = (s[i] == 'D' ? -r[h] : -c[k]);
        r[h] += g[h][k];
        c[k] += g[h][k];
        h += (s[i] == 'D');
        k += (s[i] == 'R');
    }
    g[n - 1][m - 1] = -r[n - 1];
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            cout << g[i][j] << " \n"[j == m - 1];
}

```

Secret Message

Source: Codeforces Round 1078 (1400)

Link: <https://codeforces.com/contest/2194/problem/C>

Problem Statement Bob, a genius spy, has intercepted k strips of encrypted paper. Each strip has length n and consists of strictly lowercase Latin letters.

Based on his experience, Bob hypothesizes that the true secret message (the decryption) is also a string of length n . The construction of this message follows a specific rule: the i -th letter of the message corresponds to the i -th letter of one of the intercepted strips. Bob defines the informativity of a string as the minimum positive integer d such that the string can be formed by repeating a shorter string t of length d several times.

Bob assumes the true message is the one that satisfies the construction rule and has the minimum possible informativity. Help Bob find the message.

Reasoning Notice that the constructed string has to be k -th periodic, the period of the string has to be a divisor d of n . This means that $a_i = a_{i+d} = a_{i+2d} = \dots = a_{i+md}$, the letters that we can choose in position $S(i)$ are $\bigcup_j^k s[j][i]$, so for our constructed string to be d periodic this means that $\bigcap_{i+=d}^n S(i)$, if this set is empty there's no letter that we can choose. Represent the sets as bitsets so the operation of intersection and union are as fast as possible, precompute all factor sets up to constraints for n , and check for each divisors d if it's possible to construct such d -periodic string, return the first constructed valid string since the factors are naturally sorted in an ascending order.

```

vector<int> f[50001];
int n, k;
void solve(){

```

```

cin >> n >> k;
vector<string> inp(k);
for(int i = 0; i < k; i++){
    cin >> inp[i];
}
vector<int> sts(n);
for(int i = 0; i < n; i++){
    int bs = 0;
    for(int j = 0; j < k; j++){
        bs |= (1 << (inp[j][i] - 'a'));
        sts[i] = bs;
    }
}
for(int d : f[n]){
    string ans = "";
    bool ok = true;
    for(int j = 0; j < d; j++){
        int el = -1;
        for(int i = j; i < n; i += d)
            el &= sts[i];
        if(!el){
            ok = false;
            break;
        }
        for(int i = 0; i <= 31; i++){
            if(el & (1 << i)){
                ans += (char)('a' + i);
                break;
            }
        }
    }
}

if(ok){
    string info = ans;
    while(sz(ans) != n)
        ans += info;
    cout << ans << '\n';
    return;
}
}

void precompute(){
    for(int i = 1; i <= 50000; i++)
        for(int j = i; j <= 50000; j += i)
            f[j].pb(i);
}

```

One night at freddy's

Source: Codeforces Round 996 (1600)

Link<https://codeforces.com/contest/2055/problem/C>

Problem Statement Let n, m, l be positive integers. You have made the unfortunate decision to work as a night guard at Freddy Fazbear's Pizzeria, where there are m animatronics numbered $1, \dots, m$ waiting to jumpscare you. The night consists of l seconds, numbered $1, \dots, l$. The j -th of the m animatronics has a danger level d_j , and initially $d_1 = \dots = d_m = 0$. Every second, the danger level of exactly one animatronic will increase by 1,

and throughout the night, you are able to observe all values of d_j at the current time.

You are not defenseless, however. At each of the a_i fixed times, you can shine your flashlight on exactly one animatronic j of your choice. This occurs immediately after the a_i -th second and sets its danger level back to zero, $d_j = 0$.

Let the overall danger be the maximum danger across all animatronics.. You will lose if the overall danger at the end of the night (after l seconds) is greater than x . Find the minimum value of x such that regardless of the actions of the animatronics, you can guarantee that overall danger will be less than or equal x .

Pair of balls

Source: ABC 216 (1300)

Link: <https://codeforces.com/contest/2055/problem/C>

Problem Statement We have $2N$ balls. Each ball has a color represented by an integer between 1 and N (inclusive). For each of the N colors, there are exactly two balls of that color. These balls are contained in M cylinders placed perpendicularly to the floor. Initially, the i -th cylinder ($1 \leq i \leq M$) contains k_i balls, the j -th of which from the top ($1 \leq j \leq k_i$) has color a_{ij} .

Your objective is to empty all M cylinders by repeating the following operation.

Choose two different non-empty cylinders and remove the topmost ball from each of them. Here, the two balls removed must be of the same color. Determine whether the objective is achievable.

Reasoning Notice that if at some point in time we can delete two identical balls from the top of two different cylinders then it's optimal to take them. Suppose that we have the possibility to choose colors c_{i_1}, c_{j_1} and c_{i_2}, c_{j_2} s.t. $i_1 \neq i_2 \neq j_1 \neq j_2 \wedge c_{i_1} = c_{j_1} \wedge c_{i_2} = c_{j_2}$. By deciding to remove one of the two the other will always be at disposal since it will not match with any other cylinder's top-color. It suffices to carefully implement a simulation eliminating two identical top-colors as soon as it's possible.

```
void solve() {
    int n, m, cnt = 0; cin >> n >> m;

    vector<queue<int>>> vq(m);
    map<int, vector<int>>> pos;
    queue<pair<int, int>> st;

    for (int i = 0; i < m; i++) {
        int x; cin >> x;
        for (int j = 0; j < x; j++) {
            int y; cin >> y; vq[i].push(y);
        }
    }

    auto proc = [&](int i) {
        if (!vq[i].empty()) {
            int x = vq[i].front();
            pos[x].pb(i);
            if (pos[x].size() == 2) {
                st.push({pos[x][0], pos[x][1]});
            }
        }
    };

    for (int i = 0; i < m; i++) {
        proc(i);
    }
```

```

while (!st.empty()) {
    auto [i, j] = st.front(); st.pop();
    vq[i].pop();
    vq[j].pop();
    cnt += 2;
    proc(i); proc(j);
}

cout << (cnt < 2 * n ? "No\n" : "Yes\n");
}

```

Count123

Source: ABC 456 (1600)

Link: https://atcoder.jp/contests/abc458/tasks/abc458_e

Problem Statement Count the number, modulo 998244353, of sequences $A := A_1, \dots, A_{X_1+X_2+X_3}$, that contains X_1 copies of 1, X_2 copies of 2 and X_3 copies of 3. For any index $2 \leq i \leq X_1 + X_2 + X_3$ it should hold that $|A_i - A_{i-1}| \leq 1$.

Reasoning Notice that the condition is equivalent to saying that there cannot be any 1 element and 3 element that are adjacent to each other. Start by looking at the sequence S composed only of 2 of size X_2 . Let's define $X_2 + 1$ cuts. $S := \{ |s_1|s_2|\dots|s_{X_2}| \}$ we can indeed insert any not empty block composed only of 1 element or only 3 elements in place of any of the $|$ placeholders present in S , in this way ones and threes will never 'touch'. Let's start by fixing the number BL of non empty block composed only of 1 elements There are $\binom{X_2+1}{BL}$ ways to inject these BL blocks inside S . Now we have to decide the size of these BL blocks, the number of ways to partition X_1 ones in BL non empty blocks is $\binom{X_1-1}{BL-1}$. What about the X_3 threes? we can distribute them now in the $X_2 + 1 - BL$ remaining cuts, it's not necessary for a cut to be filled with at least one three so the formula is $\binom{X_3+X_2-BL}{X_2-BL}$. Thus the formula to implement is the following.

$$\sum_{BL=1}^{X_2} \binom{X_2+1}{BL} \binom{X_1-1}{BL-1} \binom{X_3+X_2-BL}{X_2-BL}$$

```

void pre() {
    cb.init(MAXN);
}

void solve() {
    int x, y, z;
    cin >> x >> y >> z;
    Mint ways = 0;
    for (int i = 1; i <= y; i++)
        ways += cb.c(y + 1, i) * cb.c(x - 1, i - 1) * cb.c(z + (y - i), y - i);

    cout << ways << '\n';
}

```

A Ribbon For Tomorrow

Source: Codeforces Round 1116 (1600)

Link: <https://codeforces.com/contest/2255/problem/B>

Problem Statement Given a binary string $S := S_1, \dots, S_N$ composed only of zeroes and ones, we can make the following operation any number of times, possibly zero: Select two indexes $1 \leq i < j \leq N$ s.t. $S_i = S_j$ and reverse the substring $[S_i, S_j]$ in other words $S := S_1, \dots, S_{i-1}, S_j, S_{j-1}, \dots, S_{i+1}, S_i, S_{j+1}, \dots, S_N$. Count the number of achievable strings after any number of operations.

Reasoning First let's define the string S in another way, define a block of X zeroes as $B(X, 0)$ and a block of Y ones as $B(Y, 1)$. Then S is the concatenation of some number of alternating blocks of various sizes. For example $S = 1000110$ is $B(1, 1), B(3, 0), B(2, 1), B(1, 0)$. Let's now suppose the dimensions of all blocks are one, can we ever change the order of the zero blocks and the one blocks, will blocks previously disjoint become adjacent, thus merging? No the string can't change. If the blocks are of arbitrary dimensions the fundamental alternating sequence can't be changed, the operation only allows to redistribute zeroes and ones among the zero blocks and one blocks, note that each block should keep at least one element for the fundamental sequence to not change. How to distribute? Given two blocks $F = B(2, 0), G = B(2, 0)$, starting at indexes i and j respectively, by doing $op_1 := i, j + 1$ we may move one zero over from G to F . Symmetrically we can move one zero from F to G . We can redistribute leftovers zeroes and ones independently between zero blocks and one blocks respectively. Let's define the leftovers zeroes to redistribute as Z , the leftovers ones to redistribute O , the number of zero blocks as NZ , the number of one blocks as NO the formula to implement is the following.

$$\binom{Z + NZ - 1}{NZ - 1} \binom{O + NO - 1}{NO - 1}$$

```
void solve() {
    int n; cin >> n;
    string s; cin >> s;
    vector<pii> b;
    char pv = s[0];
    int cnt = 1;
    for (int i = 1; i < n; i++) {
        if (s[i] == pv) {
            cnt++;
        } else {
            b.pb({pv - '0', cnt});
            cnt = 1;
        }
        pv = s[i];
    }
    b.pb({pv - '0', cnt});
    int nz = 0, nbz = 0;
    int no = 0, nbo = 0;

    for (const auto &[i, cn] : b) {
        if (i == 0) {
            nz += (cn - 1);
            nbz++;
        } else {
            no += (cn - 1);
            nbo++;
        }
    }
    Mint oways = Mint(1), zways = Mint(1);
    if (nbz > 1 && nz >= 0) zways = cb.c(nz + nbz - 1, nbz - 1);
    if (nbo > 1 && no >= 0) oways = cb.c(no + nbo - 1, nbo - 1);
    cout << oways * zways << '\n';
}
```

Minimum Operations to Make Character Frequencies Equal

Source: Leetcode 3389 (1800)

Link: <https://leetcode.com/problems/minimum-operations-to-make-character-frequencies-equal/description/>

Problem Statement Given a string S containing only lowercase english letter. It's possible to perform the following operations any number of times:

- Delete a character from S
- Insert a character from S
- Change a character to its next letter in the english alphabet.

Note that the alphabet it's not circular hence the character 'z' can't be replaced. Determine the minimum number of operations to make all the character's frequencies equal.

Reasoning Note that the first operation and third operation are very similar for instance, we can treat the first operation as a 'regret' of the third operation. Let's say we have $n(c)$ occurrences of character c and let d be the next adjacent character in the alphabet, we can delete $m \leq n(c)$ occurrences of c and add d at most m times, notice how and this additions comes for free. Note that this condition is equivalent to transform S to a string T where the number of occurrences for each character in the alphabet is either 0 or K . Denote $f(K)$ the minimum number of operation needed to transform S in such T . Let's suppose we chose a fixed frequency of K , and let $n(c)$ the number of occurrences of the character c in the original string S , then we can differentiate the following cases:

- $n(c) \geq K$ in this case is optimal to do the first and third operation combined, this means delete $D := n(c) - K$ copies of c , as previously said these D deletions can be considered as token (or free additions) for only the next character d .
- $n(c) < K$ in this case there are two choices, we can either set $n(c)$ to 0 or K and there is no clear optimal choice. If we decided to set $n(c) \rightarrow 0$ then the same mechanism applies as there will be at most $D := n(c)$ deletions and tokens for the next character. Conversely if we decided to set $n(c) \rightarrow K$, we can decide to either consume any amount of token (if present) from the previous character or not. Note that it's useless to use more than $K - n(c)$ tokens. If after all possible free insertion it holds $n(c) < K$ then we need to insert $K - n(c)$ more.

We can model this process with dynamic programming, where $dp[c][ch]$, represent to the minimum number of operation to make all character's frequencies up to c either 0 or K and $n(c)$ as been setted according to ch . Bruteforcing through all possible K , and calculating $f(K)$ for each K finds eventually the optimal answer in $O(N)$.

```
void solve()
{
    string s; cin >> s;
    vector<int> ch(N, 0);
    int n = s.size();
    for(int i = 0; i < n; i++) ch[s[i] - 'a']++;
    int ans = accumulate(ch.begin(), ch.end(), 0);
    for(int f = 1; f <= n; f++)
    {
        vector<vector<int>> dp(N, vector<int>(2));
        dp[0][0] = dp[0][1] = (ch[0] >= f ? ch[0] - f : ch[0]);
        if(ch[0] < f) dp[0][0] = min(f - ch[0], dp[0][0]);
        for(int b = 1; b < N; b++)
        {
            dp[b][0] = dp[b][1] = (ch[b] >= f ? ch[b] - f : ch[b]) + min(dp[b-1][0], dp[b-1][1]);
            if(ch[b] < f)
            {
                int fin = (ch[b] + (ch[b-1] >= f ? ch[b-1] - f : ch[b-1]));
                dp[b][0] = min(min(dp[b-1][0] + f - ch[b], dp[b-1][1] + max(f - fin, 0)), dp[b][0]);
            }
        }
        ans = min(ans, min(dp[N-1][0], dp[N-1][1]));
    }
    cout << ans << '\n';
}
```

Matrix Game

Source: Codeforces Round 1033 (1800)

Link: <https://codeforces.com/contest/2120/problem/D>

Problem Statement Aryan and Harshith play a game. They both start with three integers a , b , and k . Aryan then gives Harshith two integers n and m . Harshith then gives Aryan a matrix X with n rows and m columns, such that each of the elements of X is between 1 and k (inclusive). After that, Aryan wins if he can find a submatrix* Y of X with a rows and b columns such that all elements of Y are equal. Aryan gives you the values of a , b , and k . He asks you to find the lexicographically minimum tuple (n, m) that he should give to Harshith such that Aryan always wins. Help Aryan win the game. Assume that Harshith plays optimally. The values of n and m can be large, so output them modulo $10^9 + 7$. A tuple (n_1, m_1) is said to be lexicographically smaller than (n_2, m_2) if either $n_1 < n_2$ or $n_1 = n_2$ and $m_1 < m_2$.

Definition A submatrix of a matrix is obtained by removing some rows and/or columns from the original matrix.

Reasoning Let's first investigate the first dimension of the matrix Harshith has to fill. It's clear that if $n \leq (a-1)k$, every column will contain $a-1$ elements for each k , so there will be never a submatrix with a rows. Assume $n := (a-1)k + 1$ in this case we know in each column there has to be at least one $1 \leq i \leq k$ that appears a times. Since there are a finite number of different columns let's call it $P = k^n$ we may set $m := P(b-1) + 1$ so n is enough (as there will be b identical columns). Notice how the set P doesn't have to contain all possible columns, we need that no two columns exist that have a elements of type $1 \leq i \leq k$ in the same exact positions. For each $1 \leq i \leq k$ we can choose $\binom{n}{a}$ positions, let's assign a column to each of this configurations, in this way no two columns of this group can have a elements of type i in the same positions. What if we decided to put one more i elements in one of the $n-a$ remaining elements of one of these columns? In this case there would be for sure two columns having the i elements in the same positions, so we must delete one of the two from this set. We deduce that we must put j elements where $j \neq i$ in the remaining spots of the columns of the i group. Notice that for each column in the group, we could assign to its $n-a$ free spots $a-1$ copies of j for each $j \neq i$, then we would be sure there would be no conflict inside and outside of this i group. To be precise if we decided to have a column with a copies of i and j with $i \neq j$ in the i group then this would conflict with some other column in the j group, we deduce that is never better to cross groups in this way. Therefore $P := \sum_{i=1}^k |S_i|$ and all the $|S_i|$ are equal thus the formula to implement is:

$$m := k \binom{n}{a} (b-1) + 1$$

```
void pre() {cb.init(MAXN);}
void solve()
{
    int a,b,k;cin>>a>>b>>k;
    ll tn=1ll*(a-1)*k+1;
    Mint n=Mint(tn);
    Mint num=Mint(1),den=Mint(1);
    for(int i=0;i<a;i++)num*=(tn-i);
    for(int i=1;i<=a;i++)den*=i;
    cout<<n<<' '<<k*(num/den)*(b-1)+1<<'\n';
}
```

Sum of Digits (and Again)

Source: Educational Codeforces Round 188 (1800)

Link: <https://codeforces.com/contest/2204/problem/E>

Problem Statement For a positive (strictly greater than 0) integer x , the string $S(x)$ is formed through the following process:

1. Initially, this string is empty.

2. The decimal representation of the number x , without leading zeros, is appended to the right of the string.
3. If $x \leq 9$, the process ends.
4. Otherwise, x is replaced by the sum of the digits of x , and the process returns to step 2.

You are given a string s consisting of digits. Your task is to rearrange the characters in this string so that it forms the string $S(x)$ for some number x . Removing characters and/or adding new characters is not allowed. If the given string s is already the string $S(x)$ for some number x , you may leave it unchanged. It is guaranteed that such rearrangement exists.

Reasoning For some integer x let's define the sequence $A = x, S(x), S(S(x)), S(S(S(x))), \dots$ we will denote $S^2(x)$ as $S(S(x))$. This sequence is guaranteed to end at some point t where $S^t(x) \leq 9$. Let's suppose the input string s is a rearrangement of a sequence where $t = 1$, for this to be true the string itself has to be that composed of one character. Let's now analyze $t = 2$, in this case the final digit is the sum of all the other digits present in the input string so if we let d be the ending digit and $T = \sum s_i$ be the sum of all digit it has to hold $T - d = d$. Let's finally analyze the case with $t \geq 3$, and let the sequence be $A = x, S(x), S^2(x), \dots, S^t(x)$, let's suppose the final ending digit is d so $S^t(x) = d$, then we know that $T = S(x) + S^2(x) + \dots + 2S^t(x)$ and let $T' = T - S^t(x)$. We can try to guess the first $S(x)$, in general it should be an integer less than T' , let's try every possible $S_g(x)$, for each guess let's generate the rest based on this first guess $S_g^2(x), \dots, S_g^t(x)$, each element of this guessed sequence has to be formed of some characters in the original string so we can never run out of characters, then $S_g^t(x) = d$ and finally $S_g(x) + S_g^2(x) + \dots + S_g^t(x) = T'$. Finally $S_g(x)$ should be the sum of all not yet assigned character (the ones not forming any $S_g^i(x)$ for all $1 \leq i \leq t$). Notice how the the guessed starting $S_g(x)$ has to be congruent modulo 9 to the final digit as x and $S(x)$ are the same modulo 9. Thanks to this the we can write a bruteforces that passes all testcases.

```
void solve()
{
    string s; cin >> s;
    int n = 0;
    vector<int> dm(10, 0);
    for(char &ch : s) n += (ch - '0');
    for(char &ch : s) dm[ch - '0']++;

    for(int d = 1; d < 10; d++) if(dm[d] > 0)
    {
        if(n == d)
        {
            cout << char(d + '0') << '\n';
            return;
        }
        if(n - d == d)
        {
            string ret = "";
            dm[d]--;
            for(int cd = d; cd >= 0; cd--) ret += string(dm[cd], char(cd + '0'));
            cout << ret + char(d + '0') << '\n';
            return;
        }
    }

    string ans = "";
    vector<int> ansdm(10, 0);
    for(int d = 1; d < 10; d++) if(dm[d] > 0)
    {
        int t = n - d;
        for(int g = t - d; g >= d; g--)
        {
```

```

    if(g%9!=d%9) continue;
    vector<int> tdm=dm;
    int c=t, t1=g;
    string ss="";
    bool ok=true;
    string f="";
    while(c>0&&ok)
    {
        ss=to_string(t1);
        for(char& ch:ss)
        {
            if(tdm[ch-'0']==0) ok=false;
            tdm[ch-'0']--;
        }
        c-=t1;
        f+=ss;
        t1=0;
        for(char& ch:ss)
        {
            t1+=ch-'0';
        }
    }

    if(c==0&&ok&&ss==to_string(d))
    {

        int k=0;
        for(int cd=0; cd<10; cd++) k+=cd*tdm[cd];
        if(k==g)
        {
            ans=f;
            ansdm=tdm;
            goto end;
        }
    }
}
end:

string ret="";
for(int i=9; i>=0; i--) ret+=string(ansdm[i], char(i+'0'));
cout<<ret+ans<<'\n';
}

```

Exquisite Array

Source: Codeforces Round 1072 (1800)

Link: <https://codeforces.com/contest/2184/problem/E>

Problem Statement We call an array of numbers k -exquisite if it has at least two elements and any two adjacent numbers differ by at least k .

You are given a permutation p of length n . For each k from 1 to $n-1$, find the number of k -exquisite subarrays.

Reasoning Let's first analyze the case for $k=1$, in this case any subarray $p[l:r]$ such that $r-l+1 \geq 2$ is going to be exquisite, since it's guaranteed there are no repeated elements in p . To count the number of subarrays of an array, we count just the number of unordered pairs, so $\binom{n}{2}$. Looking at the previous observation we can now

define an k -exquisite partition $P(k)$ of the permutation p , in blocks b_j each composed of a left boundary l_j and a right boundary r_j . Any index $1 \leq i \leq n-1$ s.t. $|a_i - a_{i+1}| < k$ act as one of the separator defining the partition $P(k)$, once the partition is constructed any subarray of the maximal blocks is k -exquisite. The formula should be $S(k) := \sum_{b_j \in P(k)} \binom{r_j - l_j + 1}{2}$, however calculating the partition $P(k)$ for each k and calculating this formula for each $P(k)$ is expensive. We can notice that $S(n)$ is zero there cannot be any block of $P(k)$ with length more than two. Let's now assume there is one index with $1 \leq i \leq n-1$ such that $|a_i - a_{i+1}| \geq n-1$, turns out we can calculate $P(n-1)$ from $P(n)$, indeed we would merge two blocks in $P(n)$ to form $P(n-1)$. This process can be seen as removing the one separator. We can therefore track this $P(k)$ with dsu data structure that can and track each block size and merge two blocks in $O(\log n)$ (with union by size) or $O(\alpha)$ (if we add path compression). Therefore everytime we can merge two blocks we delete the previous contribution of both disjoint blocks and we add the new merged contribution of the new merged block. Note that the process is monotonic from $P(n-1)$ to $P(1)$ as we can only merge blocks until we reach the already analyzed the $k=1$ case.

```
struct dsu
{
    int n;
    vector<int> l, s;
    dsu(int sz): n(sz), l(n), s(n, 1)
    {
        iota(all(l), 0);
    }

    int find(int i)
    {
        int r=i;
        while(l[r]!=r) r=l[r];
        return r;
    }

    void join(int i, int j)
    {
        int ri=find(i), rj=find(j);
        if(ri==rj) return;
        if(s[ri]<s[rj]) swap(ri, rj);
        s[ri]+=s[rj];
        l[rj]=ri;
    }

    int gets(int i)
    {
        int r=find(i);
        return s[r];
    }
};

void solve()
{
    int n; cin>>n;
    vector<int> a(n);
    vector<ll> ans(n);
    for(int &e:a) cin>>e;
    dsu d(n);
    ll ways=0;
    vector<int> pr(n-1); iota(all(pr), 0);
    sort(all(pr), [&a](int i, int j)
    {
        return abs(a[i]-a[i+1])<abs(a[j]-a[j+1]);
    });
    for(int i=n-1; i>=1; i--)
```

```

{
    while(!pr.empty() && abs(a.at(pr.back()) - a.at(pr.back()+1)) >= i)
    {
        int t = pr.back();
        pr.pop();
        int oldsa = d.gets(t), oldsb = d.gets(t+1);
        ways -= (1ll * oldsa * (oldda - 1)) / 2;
        ways -= (1ll * oldsb * (olddb - 1)) / 2;
        d.join(t, t+1);
        int news = d.gets(t);
        ways += (1ll * news * (news - 1)) / 2;
    }
    ans[i] = ways;
}
for(int i = 1; i <= n; i++) cout << ans[i] << " \n" [i == n];
}

```

AND-Array

Source: Nebius Round 2 (Codeforces Round 1088) (1900)

Link: <https://codeforces.com/contest/2211/problem/D>

Problem Statement For a sequence $a = (a_1, a_2, \dots, a_n)$ of length n , we define its $f(a)$ as follows:

$$f(a)_k = \sum_{1 \leq i_1 < i_2 < \dots < i_k \leq n} (a_{i_1} \& a_{i_2} \& \dots \& a_{i_k})$$

for all $1 \leq k \leq n$, where $\&$ denotes the bitwise AND operation.

In other words, $f(a)_k$ is the sum of the bitwise AND over all subsequences of a of length k .

The sequence a is unknown to you. However, you are provided with a sequence $b = (b_1, b_2, \dots, b_n)$ of length n such that:

$$b_i \equiv f(a)_i \pmod{10^9 + 7} \quad \text{for all } 1 \leq i \leq n$$

Your task is to reconstruct the sequence a based on the given sequence b .

Reasoning We will construct an array a such that for each pair $i < j$ s.t. $1 \leq i < j \leq n$ then it holds $a_i \& a_j = a_j$, in other words by going right to left we can only add bits until we filled all possible bits. Let's start by analyzing $f(a)_n$, this element represents all the bits common to all elements of a , let's fix $a_n = f(a)_n$ for the property that our array has to satisfy, now let's analyze $f(a)_{n-1}$ we know for sure $\binom{n-1}{n-2} = n-1$ summand of $f(a)_{n-1}$ will contain a_n and only one won't contain it therefore we can recover this term by doing $a_{n-1} = f(a)_{n-1} - (n-1)a_n$. In general by assuming that a has to be constructed this way then we can compute a_i from $a_j, a_{j+1}, \dots, a_n$ recursively, indeed each term of $f(a)_k$ is composed of $\binom{n}{k}$ summands, we can characterize each k -subsequence summand by looking at the rightmost a_{i_k} of that subsequence. Since a_{i_k} will be a submask of all a_{i_t} then that sequence AND $(a_{i_1} \& a_{i_2} \& \dots \& a_{i_k})$ will only depend on that a_{i_k} . Therefore let's count the number of subsequence of length k that end in $a_{k+1}, a_{k+2}, \dots, a_n$, the transition is

$$a_i = f(a)_i - \sum_{j=i+1}^n \binom{j-1}{k-1} a_j$$

The binomial stands for select element at index j and then choose some more $k-1$ indexes s.t. $i_1 < \dots < i_{k-1} < j$, in this way j is the rightmost for the sequence. This gives a $O(n)$ transition, computing this for all i it's too expensive. We know by construction that the set $S(i) = a_{i+1}, \dots, a_n$ will contain at most 32 distinct values for

the submask property, therefore instead of calculating the sum for each j we can calculate it for each contiguous blocks (a block in this case is a subarray $a[l : r]$ where $a_l = a_{l+1} = \dots = a_r = v$. Now the transition will look like this

$$a_i = f(a)_i - \sum_{b_j \in B} v_j \sum_{i=l_j}^{r_j} \binom{i-1}{k-1}$$

Therefore we must store all the blocks in the suffix $S(i) = \{i+1, \dots, n\}$ and for each block we keep track of $b_j = \{l_j, r_j, v_j\}$ (the left boundary, the right boundary, and the value all a 's are equal to. Then in $a_i = v_j$ we extend the left boundary $l_j := l_j - 1$ else if they are not equal we add a new block $b_{j+1} = \{i, i, a_i\}$. Finally we will optimize that nested sum with hockey-stick identity so the transition will execute at most 32 operations.

$$a_i = f(a)_i - \sum_{b_j \in B} v_j \left(\binom{r_j}{k} - \binom{l_j-1}{k} \right)$$

Exercise Why assuming a is built like this doesn't compromise any possible solution if none are found, can none be found, what would that imply? Can you find a simpler approach looking at bits?

```
struct blk
{
    Mint val;
    int l, r;
    blk(Mint val, int l, int r) : val(val), l(l), r(r) {}
};

void solve()
{
    int n; cin >> n;
    vector<Mint> b(n);
    for (int i=0; i<n; i++)
    {
        int x; cin >> x;
        b[i] = Mint(x);
    }
    vector<Mint> a(n, b[n-1]);
    vector<blk> blks;
    blks.eb(b[n-1], n-1, n-1);
    for (int i=n-2; i>=0; i--)
    {
        Mint t = Mint(b[i]);
        for (auto& [val, l, r] : blks)
        {
            t -= (cb.c(r+1, i+1) - cb.c(l, i+1)) * val;
        }
        a[i] = t;
        blk tp = blks.back();
        if (tp.val == t)
        {
            tp.l--;
            blks.ppb();
            blks.pb(tp);
        }
        else
        {
            blks.eb(a[i], i, i);
        }
    }
    for (int i=0; i<n; i++) cout << a[i] << " \n" [i==n-1];
}
```

It Just Keeps Going Sideways

Source: Codeforces Round 1096 (1700)

Link: <https://codeforces.com/contest/2227/problem/F>

Problem Statement Yousef has n columns of cubes standing side by side. The i -th column contains a_i identical unit cubes stacked vertically. Initially gravity pulls cubes downwards, so every column i contains exactly a_i cubes at heights $1, 2, \dots, a_i$.

Suddenly, gravity shifts to the right. Each cube slides horizontally as far to the right as possible. A cube cannot pass through or overlap other cubes, and it must remain at its original height. The final configuration is uniquely determined by the initial heights.

Before the gravity shift, Yousef may perform at most one operation: choose an index i and decrease a_i by 1 (i.e. remove one cube from that column). He may also choose to do nothing.

A cube's movement distance is defined as $|j - i|$, where i is its original column index and j is its column index after the gravity shift.

Find the maximum possible total movement distance (the sum of the movement distances of all remaining cubes) Yousef can achieve.

Reasoning It's clear that iterating on all cubes for each column is going to be expensive, we can try to calculate the contribution of a column to the final sum, a column contains cubes that will stack to the right after the gravity change they but can be also a foundation for other cubes that stand on the left of it. In general let us take some column i with height $a_i > 0$ the total movement that its blocks will do are exactly the number of non occupied spaces in the rectangle $R = n - i \times a_i$, each non occupied space at a height $1 \leq h \leq a_i$ will make the h -th block of the i -th row move by one. We can calculate the area of this rectangle, with a segment tree that keeps the frequency of columns on the right with height h , and another that keeps the heights themselves. The contribution formula for column i , can look like this.

$$C(i, a_i) := F(h < a_i)a_i - H(h < a_i) \quad h \in S(i)$$

Where F and H tracks frequencies and heights over the suffix $S(i) = (i, \dots, n]$. We can even calculate how much movement there would be in case we decreased a_i by one if $a_i > 0$ in this case we just compute $C(i, a_i - 1)$ at the same time. Decreasing a column height has also another effect, we can see that that cube will act as an empty space for any column in the prefix $P(i) = [1, \dots, i)$ having $h \geq a_i$. Therefore a final sweep left to right with another segment tree tracking column heights in the prefix will suffice to adjust and evaluate the possible height decreasing scores.

```
struct segtree
{
    int n;
    vector<ll>seg;
    segtree(int sz):n(sz),seg(2*n,0){};
    void upd(int i,int v)
    {
        seg[i+=n]+=v;
        for(int f=i>>1;f>=1;f>>=1)
        {
            seg[f]=seg[f<<1]+seg[f<<1|1];
        }
    }
    ll que(int l,int r)
    {
        ll t=0;
        for(int cl=l+n,cr=r+n;cl<=cr;cl>>=1,cr>>=1)
        {
            if(cl%2==1)t+=seg[cl++];
        }
    }
};
```

```

        if(cr%2==0)t+=seg[cr--];
    }
    return t;
}
};

void solve()
{
    int n;cin>>n;
    vector<int>a(n),cnt(n+1);
    vector<ll>emp(n+1),empn(n+1);
    for(int &e:a)cin>>e;
    segtree d(n+1),c(n+1),e(n+1);
    auto cem=[&](int i,int v)->ll{return c.que(0,v-1)*v-d.que(0,v-1)};
    ll tot=0;
    for(int i=n-1;i>=0;i--)
    {
        emp[i]=cem(i,a[i]);
        if(a[i]>0)empn[i]=cem(i,a[i]-1);
        d.upd(a[i],a[i]);
        c.upd(a[i],1);
        tot+=emp[i];
    }
    ll ans=tot;
    for(int i=0;i<n;i++)
    {
        if(a[i]>0)
        {
            ans=max(tot+e.que(a[i],n)-emp[i]+empn[i],ans);
            e.upd(a[i],1);
        }
    }
    cout<<ans<<'\\n';
}

```

Array Replacement

Source: Codeforces Round 1115 (1700)

Link: <https://codeforces.com/contest/2252/problem/D>

Problem Statement You are given an array a of length n .

You can perform the following operation any number of times (possibly zero): choose an index i ($2 \leq i \leq n-1$) such that a_{i-1} and a_{i+1} have the same parity. Replace a_i with $a_{i-1} - a_i + a_{i+1}$.

Find the lexicographically smallest array that can be obtained after applying any number of operations.

A sequence x is lexicographically smaller than a sequence y of the same length if and only if, at the first position where x and y differ, the element in x is strictly smaller than the corresponding element in y .

Reasoning Let's start by noticing that given some $1 < i < n$ then $a_i \equiv a_{i-1} - a_i + a_{i+1} \pmod{2}$ if $a_{i-1} \equiv a_{i+1} \pmod{2}$. Therefore the parity of an element a_i can never change. Let's suppose for the sake of simplicity that for all indexes $1 < i < n$ it holds $a_i \equiv a_{i-1} - a_i + a_{i+1} \pmod{2}$, we can notice that the a_1 and a_n can't change, while the first element we have control of is a_2 , let's start a chain of operations $(k, k-1, \dots, 2)$ at some index k s.t. $1 < i < k < n$, we can see that $a_k := a_{k-1} - a_k + a_{k+1}$ then apply an operation on $k-1$, which gives $a_{k-1} := a_{k-2} - a_{k-1} + a_k$ then by expanding a_k we get $a_{k-1} := a_{k-2} - a_k + a_{k+1}$, we can generalize this by saying that we can set $a_i := a_{i-1} + d$ where $d_j \in D(i) := \{a_j - a_{j-1} \mid i \leq j \leq n\}$, let's call this $D(i)$ set of distances present in the suffix of i . This property that we observed means more practically that we can shift left any distance, let's try to shift a distance

$d_j = a_j - a_{j-1}$ $j > 2$ from j to 2, we can observe that on the last move the first triplets changes from $a_1, a_2, a_2 + d$ to $a_1, a_1 + d, a_2 + d$, the two distances have swapped places in one operation. We indeed can permute the distances however we want therefore given the set of distances of all array $D(2)$, we sort this set in a non decreasing way and we get the permutation $d_{p_1}, d_{p_2}, \dots, d_{p_{n-1}}$, now we can apply the distances to the original sequence in this way $a_1, \dots, a_i := a_{i-1} + d_{p_{i-1}}, \dots, a_n$ (note how this is the smallest lexicographical sequence achievable). Let's now suppose $\exists! 1 < j < n$ $a_{j-1} \neq a_{j+1}$ (2), we can then partition the array in two blocks $[1, \dots, j][j, \dots, n]$ and solve each as they are independent since j acts as a roadblock. We can clearly extend this argument by partitioning a in m blocks where m is the number of roadblocks, and solve for each block.

```

struct blk
{
    int l, r;
    blk(int l, int r):l(l), r(r){}
};
void solve()
{
    int n; cin >> n;
    vector<ll> a(n);
    vector<blk> q;
    for(ll &e: a) cin >> e;
    int l = 0;
    while(l < n)
    {
        int r = l + 1;
        while(r < n - 1 && ((a[r - 1] ^ a[r + 1]) & 1) == 0) r++;

        q.eb(l, min(r, n - 1));
        l = r;
    }
    auto cmp = [](pii a, pii b)
    {
        if(a.F == b.F) return a.S > b.S;
        return a.F > b.F;
    };

    for(const auto &[l, r]: q)
    {
        priority_queue<pii, vector<pii>, decltype(cmp)> pq(cmp);
        for(int i = l + 1; i <= r; i++)
        {
            pq.push({a[i] - a[i - 1], i});
        }

        for(int i = l + 1; i < r; i++)
        {
            auto [d, j] = pq.top();
            a[i] = a[i - 1] + d;
            pq.pop();
        }
    }
    for(int i = 0; i < n; i++) cout << a[i] << " \n"[i == n - 1];
}

```

No Cost Too Great (Hard Version)

Source: Codeforces Round 1060 (2000)

Link: <https://codeforces.com/contest/2154/problem/C2>

Problem Statement You find yourself with two arrays of positive integers a and b , both of length n . You may perform the following operation any number of times (possibly none): choose an index i and increase a_i by 1. Each such increase costs b_i .

Determine the minimum total cost needed to make it possible for there to exist two indices i, j such that $1 \leq i < j \leq n$ and $\gcd(a_i, a_j) > 1$.

Reasoning Observe that if there exists already a pair (i, j) $1 \leq i < j \leq n$ s.t. $\gcd(a_i, a_j) > 1$ then the answer is zero. If there is no such pair then there can only be two optimal incremental pattern cases:

- choose two elements a_i, a_j , and try the following combinations $a_i, a_j + 1$, $a_j + 1, a_i$ and $a_j + 1, a_i + 1$, for sure one of the three combination would yield $\gcd(a_i, a_j) > 0$, it is possible thanks to a dp to calculate the minimum when we can increase one element by one or not.
- choose one element a_i and increase it more than 1 time until there exist a_j such that $\gcd(a_i, a_j) > 1$, let's find for each a_i what is the minimum number of increase of a_i ($d_i > 1$) for the property to be satisfied, then we would just calculate $\min_{1 \leq i \leq n} b_i d_i$.

The second case looks incredibly hard to calculate, we can make a couple observation to reduce this operation. Let's picture the set of all primes present $S(i)$ in some prime factorization of some a_j $j \neq i$, by increasing some element a_i , some number of times d_i we change the prime factorization of a_i , for some d_i to be valid there should be an element in the prime factorization of $a_j + d$ that already exists in $S(i)$ in other words by calling $P(a_i + d)$ the prime factorization of $a_j + d$ we want $S(i) \cap P(a_i + d) \neq \emptyset$. Let's now fix the prime p in the $S(i)$ we want the final gcd to be equal to, we notice that the distance $d_{i,p}$ it's exactly $p - (a_i \bmod p)$, therefore it suffices to find $p_i \in S(i)$ such that $p - (a_i \bmod p)$ is minimized. Checking this for each a_i is very expensive can we check only a constant number of them? Let's take into consideration the pair with the smallest (i, j) $b_i \leq b_j \leq b_{k \neq i \wedge k \neq j}$, the minimum cost to make $\gcd(a_i, a_j) > 1$ using only the first strategy would be at $\max b_i + b_j$, therefore it's useless to check if $d_j > 1$ as the cost would be $2b_j \geq b_i + b_j$, the same argument can be done for all $k \neq i$, so it suffices to check only for i .

```
const int MAXN=1e6+7;
vector<int>spf(MAXN);
void pre()
{
    iota(all(spf),0);
    for(int p=2;p*p<MAXN;p++)
        if(spf[p]==p)
            for(int j=p*p;j<MAXN;j+=p)
                if(spf[j]==j)spf[j]=p;
}
void solve()
{
    int n;cin>>n;
    vector<int>a(n),b(n);
    vector<set<int>>pf;
    for(int &e:a)cin>>e;
    for(int &e:b)cin>>e;

    map<int,int>dp;
    ll ans=IINF;
    auto getpf=[&spf](int t)->set<int>
    {
        set<int>ans;
        while(t!=1)
        {
            ans.insert(spf[t]);
            t/=spf[t];
        }
    }
```

```

    }
    return ans;
};

for(int i=0; i<n; i++)
{
    set<int> p0=getpf(a[i]), p1=getpf(a[i]+1);
    for(auto& p:p0)
    {
        if(dp.count(p)) ans=min(1ll*dp[p], ans);
        dp[p]=0;
    }
    pf.pb(p0);
    for(auto& p:p1)
    {
        if(dp.count(p))
        {
            ans=min(1ll*dp[p]+b[i], ans);
            dp[p]=min(dp[p], b[i]);
        }
        else
        {
            dp[p]=b[i];
        }
    }
}
if(ans==0) return void(cout<<0<<'\n');
int i=distance(b.begin(), min_element(all(b)));

for(int j=0; j<n; j++)
{
    if(i!=j)
    {
        for(const auto& p:pf[j]) ans=min(1ll*(p-(a[i]%p))*b[i], ans);
    }
}
cout<<ans<<'\n';
}

```

Catshock

Source: Codeforces round 1060 (2000)

Link: <https://codeforces.com/contest/2154/problem/D>

Problem Statement A cat lives on a tree with n nodes. The cat starts on node 1, and you live on node n . You are going to leave the cat a note written in the parkour language to help it reach you.

The parkour language has two types of instructions:

- 1 — This means that the cat should move to any adjacent node to it. If there are multiple options, it will pick one of them arbitrarily. If there are no adjacent nodes to it, then it will not move.
- $2u$ — This means to destroy node u and all adjacent edges to it. If the cat is currently on node u , it will die, so this should be avoided. If node u was already destroyed, then nothing will happen.

Additionally, there cannot be two consecutive instances of the second instruction. Unfortunately, the parkour language is ambiguous because the cat may have multiple options for each instance of the first instruction. So you should construct a sequence of instructions of length at most $3n$ so that if the cat follows them, it will end at node n , no matter what choices it makes. It can be proven that such a sequence exists for any tree.

Reasoning To begin, let's worry about not kill the cat by deleting a node where the cat is currently on, tracking the cat looks impossible, one thing that might be useful, is to track the number of step of taken from node 1, in particular we could treat this number as a maximum distance D , a node that is more distant from 1 then D at time t can be safely deleted, what about star graphs centered in vertex 1? This idea looks useless in trees where the diameter is very small, in this case after one step we don't really know if we can safely delete anything, turns out we can look at the parity of the number of steps taken, therefore by calling D the number of steps taken and v an hypothetical node where the cat could be in, it holds that $D \equiv d(u, v) \pmod{2}$, where $d(u, v)$ it's the distance between two vertices (unique and minimal in trees). How to direct the cat to node n now? It helps us a recurrent idea, let's assign to each vertex $d(v, n)$, the path $p(u, n)$ from any vertex u to vertex n at some point has to converge to the vertex n with $d(n, n) = 0$, suppose we delete a vertex v that has distance $d(v, n) < d(u, n)$ and suppose the cat sits at vertex u , what happens? there can be two cases:

- $v \in p(u, n)$ in this case it will be impossible for the cat to reach n from u .
- $v \notin p(u, n)$ we got lucky, the path is still valid.

Let's assume that if $1 \leq u, v \leq n$ $d(u, n) > d(v, n)$ then u has to be deleted before v in the desired sequence, if this property holds we will always fall in the second case. We got a winning recipe, sort the vertices by non decreasing distance $d(u, n)$, and track the parity of the number of steps done since the beginning, if the vertex to delete has the same parity, we will make the cat move by one and then delete it, if they have different parity, by the 'no two deletions constraint' we will make one step for the constraint and another to change the parity then we'll delete it. Notice how we don't need to delete n , since it would kill the cat in the end, therefore there can be at max $3(n - 1)$ instructions which comfortably fits in the maximum.

```
const int MAXN=200200;
vector<int>g[MAXN];
void solve()
{
    int n;cin>>n;
    for(int i=1;i<=n;i++)g[i].clear();
    for(int i=1;i<=n-1;i++)
    {
        int u,v;cin>>u>>v;
        g[u].pb(v);
        g[v].pb(u);
    }
    vector<int>vs(n);iota(all(vs),1);
    vector<int>pr(n+1),dis(n+1);
    vector<bool>visp(n+1),visd(n+1);
    pr[1]=0;dis[n]=0;visd[n]=visp[1]=true;
    auto dfsp=[&](auto&& sf,int u)->void
    {
        for(auto& f:g[u])
        {
            if(!visp[f])
            {
                visp[f]=true;
                pr[f]=pr[u]^1;
                sf(sf,f);
            }
        }
    };
    auto dfsd=[&](auto&& sf,int u)->void
    {
        for(auto &f:g[u])
        {
            if(!visd[f])
            {
                visd[f]=true;
```

```

        dis[f]=dis[u]+1;
        sf(sf,f);
    }
}
};
dfsp(dfsp,1);
dfsd(dfsd,n);
sort(all(vs),[&dis](int i,int j)
{
    return dis[i]>dis[j];
});
assert(vs[n-1]==n);
vs.ppb();
vector<string>ans;
int curp=0;
for(const auto& v:vs)
{
    if(pr[v]==curp)
    {
        ans.pb("1");
        ans.pb("2 "+to_string(v));
        curp^=1;
    }
    else if(pr[v]!=curp)
    {
        ans.pb("1");
        ans.pb("1");
        ans.pb("2 "+to_string(v));
    }
}
cout<<sz(ans)<<'\\n';
for(const auto&str:ans)
{
    cout<<str<<'\\n';
}
}

```

Binary Wine

Source: Codeforces round 1064 (2100)

Link: <https://codeforces.com/contest/2165/problem/C>

Problem Statement You are given n integers $a_1, a_2, \dots, a_n$ within the range $[0, 2^{30})$.

You can spend 1 coin to increase any a_i by 1. You can perform this operation any number of times.

You need to solve q queries; for each query, you are given an integer c , also in the range $[0, 2^{30})$. You would like it if there exists a sequence b of length n with the following properties:

- For every $1 \leq i \leq n$, $0 \leq b_i \leq a_i$.
- $b_1 \oplus b_2 \oplus \dots \oplus b_n = c$, where $\oplus$ denotes the bitwise XOR operation.

Please calculate the minimum number of coins you will have to spend, such that there exists a suitable sequence b .

The queries are independent, meaning that any operations you perform on the sequence a will not impact future queries.

Reasoning Let's analyze the integer c for each query $1 \leq j \leq q$, all the bits that appear in c have to appear an odd number of times in each element of the sequence, note how it's optimal to make it appear just one time, this is true since we can choose b_i freely as long as it holds $b_i \leq a_i$, as a short proof we may say that we need just 1 copy of each bit present in c in the whole b sequence, if we know such bit is set for some $b_i \leq a_i$ then we won't set it for any $b_{j \neq i} \leq a_{j \neq i}$, this is always good since we don't need to add anything to each $a_{j \neq i}$, at max we subtract some 2^k if set, thus not creating any conflict with the respective upperbounds. Let's now assign all the bits set in c to some elements $B := \{b_{i_1}, b_{i_2}, \dots, b_{i_k}\}$ having upperbounds $U := \{a_{i_1}, a_{i_2}, \dots, a_{i_k}\}$. We will try to assign the bits from highest to lowest, as it is a recurrent idea that may be good in this bit assignments problems. What can we say about the most significant bit, 2^{msb} , there has to be some $b_i \leq a_i$ that has that bit set, ultimately $a_i \geq 2^{msb}$ has to hold for us to choose a nice b_i , intuitively we would like to choose the maximum upperbound a_m $1 \leq m \leq n$, if $a_m \geq 2^{msb}$ there is no reason to increase so we assign that bit to that b_m , notice how a_m may have more space for other bits let's take in consideration for example $a_m = 2^{msb} + 2^{msb-k}$, in this case there is 2^{msb-k} space left and we may continue using it, conversely if $a_m < 2^{msb}$ we have to increase a_m to 2^{msb} which costs $2^{msb} - a_m$ moreover we can see there is no more space in a_m in this case thus we may keep using index m only if there is no other index with more space. The greedy property is that if we need to create space for some bit 2^k , we need to get the index with the most space available so the eventual increase it's minimized. This greedy property is necessary only when we need to create more space, if the current index already contains enough space for a bit to be added then we may use it. The last statement may be proved in the following way: Imagine that the current analyzed index has space for bit 2^k , and 2^k is the maximum bits along the ones still to assign, therefore if it fits, there can be two cases

- $\exists_{x \neq m} a_x > a_m$ in this case if it fits in m , then x will fill all the other bits.
- $\nexists_{x \neq m} a_x > a_m$ in this case m is the maximum and may be the only one which doesn't cause one increase, so it's optimal to keep using m .

The time complexity of this simulation is $\mathcal{O}(29 \log_2(29))$ per query.

```
void solve()
{
    int n,q;cin>>n>>q;
    vector<int>a(n);
    for(int &e:a)cin>>e;
    sort(all(a)); reverse(all(a));
    for(int i=0;i<q;i++)
    {
        int c;cin>>c;
        int t=a[0],j=0,cs=0;
        priority_queue<int>res;

        for(int b=31;b>=0;b--)
        {
            if(c&(1ll<<b))
            {
                if(t>=1ll<<b)
                {
                    t-=1ll<<b;
                    b--;
                }
                else
                {
                    if(j<n-1||!res.empty())
                    {
                        int nxt=(j==n-1?-1:a[j+1]),nxtr=(res.empty()?-1:res.top());
                        if(nxt>nxtr&&t<nxt)
                        {
                            res.push(t);
                            t=a[++j];
                            continue;
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
    if(nxtr>=nxt&& t<nxt)
    {
        res.push(t);
        t=res.top();
        res.pop();
        continue;
    }
}
cs+=((1ll<<b)-t);
t=0;
b--;
}
continue;
}
b--;
}
cout<<cs<<'\\n';
}
}

```

Far Cities

Source: Codeforces Round 1118 (1600)

Link: <https://codeforces.com/contest/2258/problem/C>

Problem Statement This is an interactive problem. There is a hidden tree with n vertices. You can make queries with the following properties:

- Choose two vertices $1 \leq u, v \leq n$ and an integer $0 \leq d \leq n$.
- The grader responds with 1 if $\text{dist}(u, v) \geq d$ and 0 otherwise, where $\text{dist}(u, v)$ denotes the distance between vertices u and v in the tree.

Your task is to determine the length of the tree's diameter and any pair of nodes that are this distance apart. You may ask at most $3n$ queries.

Reasoning For the diameter holds a unique property, by fixing a node u , the node m that maximizes the distance $m = \max_{v \in V} d(u, v)$ has to be one of the endpoints of one diameter of the tree. Therefore if we find this m , we can run this procedure again (fixing m this time) to get the the other endpoint h . How many queries are we making? To find the element which maximizes the distance m for some node u we iterate over all nodes and ask if $\forall_{v \neq u} d(u, v) \geq c + 1$ where c is the current maximal. There can be two responses:

- YES: We investigate further on that node v thus we set $m := v, c := c + 1$ and we repeat the question.
- NO: we go on to the next node.

Let's analyze the worst case of a line graph (maximal diameter), in this case since the current maximal distance can only go up we make in total $3n - 3$ queries.

```

int ask(int u, int v, int d)
{
    cout<<"? "<<u<<' '<<v<<' '<<d<<endl;
    int r;cin>>r;
    return r;
}
void solve()
{
    int n;cin>>n;
    int mxv=1,maxd=0;

```

```

for(int v=2; v<=n; v++)
{
    for(int d=maxd+1; d<=n-1; d++)
    {
        if(ask(1, v, d)==1)
        {
            maxd=d;
            mxv=v;
        }
        else
        {
            break;
        }
    }
}
int smxv=1;
for(int v=1; v<=n; v++)
{
    if(v!=1&&v!=mxv)
    {
        for(int d=maxd+1; d<=n-1; d++)
        {
            if(ask(mxv, v, d)==1)
            {
                smxv=v;
                maxd=d;
            }
            else
            {
                break;
            }
        }
    }
}
cout<<"! " <<mxv<<' ' <<smxv<<' ' <<maxd<<endl;
}

```

Copy String

Source: Codeforces Global Round 30 (1800)

Link: <https://codeforces.com/contest/2164/problem/D>

Problem Statement Given two strings s and t of length n , your goal is to transform s into t through a series of operations. In a single operation, you construct a new string s' of length n satisfying the following conditions:

- $s'_1 = s_1$
- For each $1 < i \leq n$, s'_i can be set to either s_i or s_{i-1} .
- Replace s with s' .

Your task is to achieve this transformation using the minimum number of operations and to output the constructed string s' after each operation. If the transformation cannot be achieved in $k_{\max}$ or fewer operations, output -1 .

Reasoning Let's analyze each character of t , how many moves do we need to do to put character t_i at position i starting from s , a strategy could be to check what's the index p_{t_i} of the rightmost occurrence of t_i in $s[1 : i]$, from there we know that the number of moves to adjust t_i should be $i - p_{t_i}$, if we calculate this across all the t_i 's we may think that the total cost is $k := \max_{t_i} i - p_{t_i}$, this is indeed true since in one move we can shift by one more

than one occurrences. Let's now see if picking the rightmost occurrence works (it would be nice since it would for sure minimize k). Let's look at the following testcase

$$s := abcaaaa \quad t := aaaaabc$$

Here the strategy fails, because if we assigned $p_{t_5} = 5$, we would lose characters b and c , therefore the transformation may become unfeasible using this strategy. In general s can be transformed in t if and only if we can generate a sequence $p_{t_1}, \dots, p_{t_n}$ $t_i = s_{p_{t_i}}$, and $\forall_{i < j} p_{t_i} \leq p_{t_j}$. We may now binary search the best k . Note that for each t_i we can look at most k position before i . It's also important to take the leftmost occurrence so that we minimize the risk of losing characters as shown in the aforementioned testcase. Once we find the right k , we have to select all the p_{t_i} as implemented and checked in the binary search. The construction of the cyclical shifts won't be discussed as it is very straightforward once the p_{t_i} are chosen.

```
void solve()
{
    int n,k;cin>>n>>k;
    string s,t;cin>>s>>t;
    int lo=0,hi=k,ans=-1;
    auto can=[&](int v)
    {
        int bnd=-1;
        for(int i=0;i<n;i++)
        {
            char cur=t[i];
            int lst=-1;
            for(int j=i,c=0;j>=0&&c<=v&&j>=bnd;j--,c++)
            {
                if(s[j]==cur)
                {
                    lst=j;
                }
            }
            if(lst==-1)return false;
            bnd=lst;
        }
        return true;
    };
    while(lo<=hi)
    {
        int mi=lo+(hi-lo)/2;
        if(can(mi))
        {
            ans=mi;
            hi=mi-1;
        }
        else
        {
            lo=mi+1;
        }
    }
    cout<<ans<<'\\n';
    int bnd=-1;
    vector<int>lst(n);
    for(int i=0;i<n;i++)
    {
        char cur=t[i];
        for(int j=i,c=0;j>=0&&c<=ans&&j>=bnd;j--,c++)
        {
            if(s[j]==cur)
```

```

        {
            lst[i]=j;
        }
    }
    bnd=lst[i];
}
for(int k=1;k<=ans;k++)
{
    string snw=s;
    for(int i=0;i<n;i++)
    {
        if(lst[i]!=i)
        {
            snw[i]=s[i-1];
            lst[i]++;
        }
    }
    s=snw;
    cout<<s<<'\n';
}
}

```

Penchick and Desert Rabbit

Source: Codeforces Round 987 (1700)

Link: <https://codeforces.com/contest/2031/problem/D>

Problem Statement There are n trees arranged in a line, where the i -th tree has a height of a_i . A desert rabbit starts at some tree and can jump from the i -th tree to the j -th tree if exactly one of the following conditions holds:

- $j < i$ and $a_j > a_i$: The rabbit can jump backward to a taller tree.
- $j > i$ and $a_j < a_i$: The rabbit can jump forward to a shorter tree.

For each i from 1 to n , determine the maximum height among all trees reachable by the rabbit if it starts from the i -th tree.

Reasoning Let's take into consideration the maximum cartesian tree of the array a , by looking at the root (the maximum element a_m having index m), we may notice that all the elements sitting in the right side can jump to the maximum, so $ans_m = a_m$ and $\forall_{j>m} ans_j = ans_m$. It remains to solve the left side of the tree as the right side is already solved. The left side tree has as root $a_{m_2} \leq a_m$, once again everything to the right of m_2 can jump to a_{m_2} . Notice though that if we could jump from a_{m_2} to the right side of a_m then $ans_{m_2} = ans_m$ and $\forall_{m_2 < j < m} ans_j = ans_{m_2}$. By denoting as r the root of the current left subtree, $f(r)$ the father of r , and $l(r)$ the farthest index to the right we may make from r , then if $l(r) \geq f(r)$ then $ans_r = ans_{f(r)}$ else $ans_r = a_r$, after that the right side will follow $\forall_{r < j < f(r)} ans_j = ans_r$, once again we recurse on the left side until it's empty. I chose to calculate both with a max segment tree, but implementation may differ.

```

struct segtree
{
    int n;
    vector<int> seg;
    segtree(int sz):n(sz),seg(2*sz){}
    void upd(int i,int v)
    {
        seg[i+=n]=v;
        for(int f=i>>1;f>=1;f>>=1)

```

```

    {
        seg[f]=max(seg[f<<1], seg[f<<1|1]);
    }
}
int que(int l, int r)
{
    int id=0;
    for(int cl=l+n, cr=r+n; cl<=cr; cl>>=1, cr>>=1)
    {
        if(cl%2==1) id=max(seg[cl++], id);
        if(cr%2==0) id=max(seg[cr--], id);
    }
    return id;
}
};
void solve()
{
    int n; cin>>n;
    vector<int>a(n+1), rml(n+1), lmost(n+1), seen(n+1);
    for(int i=1; i<=n; i++) cin>>a[i];
    for(int i=n; i>=1; i--) lmost[a[i]]=i;
    segtree d(n+1), m(n+1);
    for(int i=1; i<=n; i++)
    {
        m.upd(i, a[i]);
    }
    for(int i=n; i>=1; i--)
    {
        if(!seen[a[i]])
        {
            d.upd(a[i], i);
            seen[a[i]]++;
        }
        rml[i]=d.que(1, a[i]-1);
    }

    int l=1, r=distance(a.begin(), max_element(a.begin()+1, a.end()))-1;
    while(l<=r)
    {
        int mxl=m.que(1, r);
        int t=lmost[mxl];
        if(rml[t]>r) a[t]=a[r+1];
        r=t-1;
    }
    for(int i=2; i<=n; i++)
    {
        a[i]=max(a[i-1], a[i]);
    }

    for(int i=1; i<=n; i++)
    {
        cout<<a[i]<<" \n"[i==n];
    }
}

```

Non Prime Tree

Source: Codeforces Round 992 (1900)

Link: <https://codeforces.com/contest/2040/problem/D>

Problem Statement You are given a tree with n vertices. You need to construct an array $a_1, a_2, \dots, a_n$ of length n , consisting of unique integers from 1 to $2n$, such that for each edge $u_i \leftrightarrow v_i$ of the tree, the value $|a_{u_i} - a_{v_i}|$ is not a prime number. Find any array that satisfies these conditions, or report that there is no such array.

Reasoning Let's notice the fact that we have at disposal $2n$ numbers not n , by assigning to vertices all elements with same parity then each difference will be even $\forall (i,j) \in E |a_i - a_j| = 2k \quad 1 \leq k \leq n-1$, we may notice that along all of this differences, the only one which is also prime is 2, therefore can we make sure that we assign to nodes all elements of the same parity and at the same time make this hold $\nexists (i,j) \in E |a_i - a_j| = 2$. We can prove it's impossible. Can we construct the tree such that the number of this differences is minimized? I will now show a construction that achieves at max one (i,j) s.t. $|a_i - a_j| = 2$. Start by calculating the distance $d(v, 1)$ for each node v , (the rooting may be arbitrary in this case i chose 1). We process the vertices in non-decreasing order of their distance from node 1 or the arbitrary root. Construct a list of all element in the interval $[1, 2n]$. For each node, if the distance is even, we will assign the first odd from the beginning of the list else we will pop it from the end of the list. We can prove that with this construction only the last node processed l and his father $f(l)$ will have a distance of $|a_l - a_{f(l)}| = 2$. Therefore it's enough to change a_l in the following way $a_l := a_{f(l)} + 1$. Since we took all the odds $a_{f(l)} + 1$ will always be at disposal as it is even (so never used), it is inbound $[1, 2n]$, moreover l is a leaf so it's connected to only one node, therefore we don't have to worry about any other difference.

```
const int MAXN=200200;
vector<int>g[MAXN];
void solve()
{
    int n;cin>>n;
    for(int i=1;i<=n;i++)g[i].clear();
    for(int i=1;i<=n-1;i++)
    {
        int u,v;cin>>u>>v;
        g[u].pb(v);
        g[v].pb(u);
    }

    vector<int>ans(n+1),dep(n+1),vis(n+1);
    vis[1]=1;dep[1]=0;
    auto dfs=[&](auto&& dfs,int u)->void
    {
        for(auto& f:g[u])
        {
            if(!vis[f])
            {
                dep[f]=dep[u]+1;
                vis[f]=true;
                dfs(dfs,f);
            }
        }
    };
    dfs(dfs,1);
    deque<int>odds;
    for(int u=1;u<=2*n;u+=2)
    {
        odds.pb(u);
    }
```

```

vector<int>ord(n);iota(all(ord),1);
sort(all(ord),[&dep](int i,int j){
    return dep[i]<dep[j];
});
int lst=ord.back();ord.ppb();
for(int &nd:ord)
{
    if(dep[nd]%2==0)
    {
        ans[nd]=odds.front();
        odds.ppf();
    }
    else
    {
        ans[nd]=odds.back();
        odds.ppb();
    }
}
ans[lst]=ans[g[lst][0]]+1;
for(int i=1;i<=n;i++)cout<<ans[i]<<" \n"[i==n];
}

```

Adjusting Drones

Source: Codeforces Round 1066 (1900)

Link: <https://codeforces.com/contest/2157/problem/E>

Problem Statement You are managing a fleet of n drones, each with an energy level $a_1, a_2, \dots, a_n$. You are also given a positive integer k , which represents the maximum number of drones allowed to share the same energy level.

To prevent overloads, the drones automatically perform energy balancing operations as follows. While there exists an energy level that appears strictly more than k times, they execute the following steps:

1. Every drone i whose energy a_i appears earlier (that is, there exists $j < i$ with $a_j = a_i$) is marked.
2. For each marked drone, its energy is increased by 1 unit.
3. The marks are removed.

The process stops once no energy level appears more than k times. Determine how many energy balancing operations will be performed.

Reasoning Let's start by noticing that the position of the drones does not matter as this operation just cares about the number of drones with energy levels e where $1 \leq e \leq 2n$. Let's try to look at this process bottom up, for example, let's look at the number of drones with energy level 1, if they are less or equal than k else $\text{cnt}(1) - k$ will get marked (in this case we made one operation). Notice that we may look at this process synchronously, indeed when we get at level $e = 2$ we do not care if in the previous operation some 2 element got marked and increased, we can look as it they are still here. Since this observation is not really trivial and it is central, i left a little proof that can make sense of this.

Consider the fact that if some drones of energy level 2 were marked in the previous operation this means $T_2 := \text{cnt}(2) - 1 \geq k$ gone to the next level while one 2 has remained, now some number $T_1 := \text{cnt}(1) - 1 \geq k$ from 1 rose to two, but since one drone of energy level 2 is guaranteed to be still here, all the ones that were marked will get marked as well, so we can look as the twos and the ones are rising continuously even if the 2's moved before. Conversely when the upper moving elements will block we will arrive behind and then we may rise all together if

the sum it's bigger than k .

So we can keep a quantity of elements T that represents all the elements that are rising from lower energy levels, then for each energy level c we synchronously add all the drones $T := T + cnt(c)$. If $T \leq k$ then they all finally stop and $T := 0$, this is time we when a block of rising elements tracked by T stops. Since all blocks are independent the maximum of this single timers is the answer. If $T > k$ the block keeps rising so the timer must go up by one, we also need to assure that one element stays at elements at energy level c hence $T := T - 1$.

One final edge case to pay attention to is that once we finished processing all the $2n$ energy levels there still might be a moving block $T \geq k$ with a timer associated to it. We can trivially calculate the stopping time since there are no more drones above $t := t + T - k$.

```
void solve()
{
    int n,k;cin>>n>>k;
    vector<int>cnt(2*n+1);
    for(int i=0;i<n;i++)
    {
        int x;cin>>x;cnt[x]++;
    }
    ll tot=0,c=0,ans=0;
    for(int j=1;j<=2*n;j++)
    {
        tot+=cnt[j];
        if(tot<=k)
        {
            ans=max(c,ans);
            tot=0;
            c=0;
        }
        else
        {
            tot--;
            c++;
        }
    }
    if(tot!=0)
    {
        c+=tot-k;
        ans=max(c,ans);
    }
    cout<<ans<<'\\n';
}
```

A/AB Insertion

Source: AtCoder Beginner Contest 473 (1700)

Link: https://atcoder.jp/contests/abc473/tasks/abc473_f

Problem Statement You are given a string S of length N consisting of A and B. Process a total of Q queries of the following two types:

- **Type 1:** 1 i c
Change the i -th character of S to c .
- **Type 2:** 2 l r
Let string T be the string obtained by extracting the l -th through r -th characters of the current string S . If it is possible to obtain string T by the following operation, output **Yes**; otherwise, output **No**.

Starting from the empty string, perform the following two operations any number of times, possibly zero, in any order:

- Choose any position in the string (possibly the beginning or the end) and insert A there.
- Choose any position in the string (possibly the beginning or the end) and insert AB there.

Reasoning Let's first start by defining when any string can be constructed from an empty string by doing the aforementioned operation. We may notice that the number of occurrences of characters A has to be greater or equal than the number of occurrences of characters B, moreover any B in any constructed string at any time must have a A counterpart that lies in the prefix. Let's associate with the map $b(s_i)$ the number -1 to the character B, and the number 1 to character A, a string good if for any $1 \leq i \leq |S|$, it holds $\sum_{j=1}^i b(s_j) \geq 0$, this assures that there are enough A's to match with the B's. For each substring $s[l : r]$ we are asking what is the minimum element in the prefix sum array built on top of the sequence $b[l : r] = b(s_l), \dots, b(s_r)$, this can be computed with a divide and conquer process, we may think of a segment tree since an update operation is also defined. We may store in each node of the segment tree $n(l : r)$ the sum of all elements $sn(n_i)$ and the minimum prefix sum element $mp(n_i)$ By defining $l(n_i)$ the left child and $r(n_i)$ the right child we may write the transition:

$$\begin{aligned} sn(n_i) &:= sn(l(n_i)) + sn(r(n_i)) \\ mp(n_i) &:= \min(mp(l(n_i)), sn(l(n_i)) + mp(r(n_i))) \end{aligned}$$

Now to query the maximum prefix in a range, we are going to trim the left node pointer and right node pointers, for each trim (read a reference on how bottom up segment tree works if it's unclear) we need to keep the reference to the original leftside of the array (the left side), same thing when we trim from the right side, so we keep a subfix sum (for the right pointer) initialized as the sum of all elements in the range that decreases by each right pointer trim, and a prefix sum initialized as zero that grows every time we trim from the leftside. The point update at index i is quite straightforward as $A \rightarrow B$ updates the balance by -2 at point i while $B \rightarrow A$ updates it by $+2$.

```
struct segtree
{
    int n;
    vector<int>segp, segm;
    segtree(int sz):n(sz), segp(2*n, 0), segm(2*n, 0){}
    void upd(int i, int v)
    {
        i+=n;
        segp[i]+=v, segm[i]+=v;
        for(int f=i>>1; f>=1; f>>=1)
        {
            segp[f]=segp[f<<1]+segp[f<<1|1];
            segm[f]=min(segm[f<<1], segp[f<<1]+segm[f<<1|1]);
        }
    }
    int quesum(int l, int r)
    {
        int cum=0;
        for(int cl=l+n, cr=r+n; cl<=cr; cl>>=1, cr>>=1)
        {
            if(cl%2==1) cum+=segp[cl++];
            if(cr%2==0) cum+=segp[cr--];
        }
        return cum;
    }
    int que(int l, int r)
    {
        int pf=0, suf=quesum(l, r), res=0;
        for(int cl=l+n, cr=r+n; cl<=cr; cl>>=1, cr>>=1)
        {
            if(cl%2==1)
```

```

        {
            res=min(res,pf+segm[cl]);
            pf+=segp[cl];
            cl++;
        }
        if(cr%2==0)
        {
            suf-=segp[cr];
            res=min(res,suf+segm[cr]);
            cr--;
        }
    }
    return res;
}
};
void solve()
{
    int n;cin>>n;
    string s;cin>>s;
    int q;cin>>q;
    segtree d(n);
    for(int i=0;i<n;i++)
    {
        d.upd(i,(s[i]=='A'?1:-1));
    }
    for(int i=0;i<q;i++)
    {
        int x;cin>>x;
        if(x==1)
        {
            int j;char c;cin>>j>>c;j--;
            if(s[j]==c)continue;
            if(s[j]=='A')d.upd(j,-2);
            else d.upd(j,+2);
            s[j]=c;
        }
        else
        {
            int l,r;cin>>l>>r;l--,r--;
            if(d.que(l,r)<0)cout<<"No\n";
            else cout<<"Yes\n";
        }
    }
}
}

```

Bitwise OR Game

Source: AtCoder Regular Contest 208 (1800)

Link: https://atcoder.jp/contests/arc208/tasks/arc208_a

Problem Statement There are n piles of stones, and the i -th pile ($1 \leq i \leq n$) has a_i stones. Alice and Bob will play the following game using these piles:

- Starting with Alice, the two players take turns alternately.
- In each turn, the player chooses one non-empty pile and removes one or more stones from that pile. However, the player cannot make a move that changes the bitwise OR of the numbers of stones in all piles.

- The player who cannot make a move loses.

Determine which player wins when both players play optimally.

Reasoning Let's take into consideration the game of nim as the deletion rules are the same, the only thing different is the final base case. Indeed in nim if $X := \bigoplus_{i=1}^n a_i \neq 0$ then we are in a winning position as we can always make $X = 0$ in one move, while in a losing position there's no chance we can keep X to zero. So it's impossible for a player in a winning position to end up in a situation where all the piles are zero, since that configuration's xorsum would be zero, thus a contradiction as we can never end in such situation starting from a xorsum different from zero. Let's now consider the bit a bit orsum $O := \text{OR}_{i=1}^n a_i$. We know that we can't make a move when each bit of O appears just one time, in this case note that $X = O$ (this is our base case). If $X = O$ then every move we make will make $X \neq O$, this is true since by doing an operation on any non-zero pile we will change the parity of at least one bit. If $X \neq O$ then there has to be some set of bits that appear an even non zero number of times $B := b_1, b_2, \dots, b_k$ $b_i \in O$. Consider the highest bit in this set, since we know both that the parity of this bit is now even and the number of piles where this bit is set is non zero then there is at least two piles where this bit is set. Let's choose one of these piles and adjust all the parities of all the bits in B so that they will be all odds, in this way for the next player it will hold $X = O$.

```
void solve()
{
    int n; cin >> n;
    vector<int> a(n);
    for(int &e : a) cin >> e;
    int xrt=0, ort=0;
    for(int i=0; i<n; i++) ort |= a[i];
    for(int i=0; i<n; i++) xrt ^= a[i];
    cout << (xrt != ort ? "Alice\n" : "Bob\n");
}
```

Many LCMs

Source: AtCoder Beginner Contest 445 (1700)

Link: https://atcoder.jp/contests/abc445/tasks/abc445_e

Problem Statement You are given a sequence of positive integers $a = (a_1, a_2, \dots, a_n)$ of length n . For each $k = 1, 2, \dots, n$, find the remainder when the least common multiple of the $n - 1$ elements of a excluding a_k is divided by 998244353.

Reasoning Let's work directly in $\mathbb{F}_{998244353}$, it's clear that we can't compute the $\text{lcm}(a, b)$, using classic euclidean algorithm since two equal elements in $\mathbb{F}_p$ may be different in some ring $\mathbb{Z}/n\mathbb{Z}$ such that $n \neq p$. Therefore euclidean algorithm is unfeasable. Let's use the definition of $\text{lcm}(a, b)$ based on the fundamental theorem of arithmetic, given P_a the prime factorization without multiplicities of a and P_b the prime factorization of b without multiplicities, then call M_a the multiplicities of prime p in the prime factorization of a , symmetrically we define M_b , note that if a prime p is not present in P_a , then $M_a(p) = 0$.

$$L := \text{lcm}(a_1, \dots, a_n) = \prod_{p \in \bigcup_{i=1}^n P_{a_i}} p^{\max_{i=1}^n M_{a_i}(p)}$$

So compute the prime factorizations of each element, then calculate L . To calculate each L_k notice how each element may affect L . In particular if a_k has maximum multiplicities $m(p)$ for primes $D_k := \{p_1, p_2, \dots, p_m\}$ along all multiplicities of such primes in all prime factorizations, then for each $p \in D_k$ we would have to switch to its second maximum multiplicity $sm(p)$ therefore the formula to implement is this:

$$L_k := \frac{L}{\prod_{p \in D_k} p^{m(p) - sm(p)}}$$

```

const int MAXN=1e7+1;
vector<int>spf(MAXN);
void pre()
{
    iota(all(spf),0);
    for(int i=2;i*i<MAXN;i++)
        if(spf[i]==i)
            for(int j=i*i;j<MAXN;j+=i)
                if(spf[j]==j) spf[j]=i;
}
void solve()
{
    int n;cin>>n;
    map<int,int>mx,smx;
    vector<map<int,int>>pfs(n);
    for(int i=0;i<n;i++)
    {
        int t;cin>>t;
        while(t!=1)
        {
            pfs[i][spf[t]]++;
            t/=spf[t];
        }
        for(const auto& [p,m]:pfs[i])
        {
            if(m>mx[p])
            {
                smx[p]=mx[p];
                mx[p]=m;
            }
            else if(m>smx[p])
            {
                smx[p]=m;
            }
        }
    }
    Mint ans=1;
    for(const auto&[p,m]:mx)
    {
        ans*=Mint(p).pow(m);
    }
    for(int i=0;i<n;i++)
    {
        Mint curans=ans;
        for(const auto&[p,m]:pfs[i])
        {
            if(m==mx[p])
            {
                curans/=Mint(p).pow(mx[p]-smx[p]);
            }
        }
        cout<<curans<<'\\n';
    }
}

```

Carrot Chopdown (Hard Version)

Source: Codeforces Round 1118 (1700)

Link: <https://codeforces.com/problemset/problem/2258/B2>

Problem Statement Alp loves carrots. Since he hasn't eaten lunch yet, he wants to buy a carrot salad to eat outside. However, he has a weird obsession: all the carrots must be the exact same length; otherwise, the salad doesn't look aesthetically pleasing to him. Since he is in the middle of the street and doesn't have a knife, he can't cut the carrots himself. You need to divide all the carrots using your machine and sell them to Alp.

You are given n delicious carrots with sizes $a_1, a_2, \dots, a_n$. You are also given a cutting machine, which works as follows:

For each operation:

1. You choose a set of carrots (you can choose chopped carrots again) and a positive integer x (not necessarily the same for each operation).
2. Consider every chosen carrot of length l . If $l \leq x$, this carrot is unaffected; otherwise, it is divided into two carrots of sizes x and $l - x$.

We'll sell some of the final carrots to an interesting guy who wants them all to be the same length.

We are asking you to determine the maximum number of carrots we can sell after using this machine exactly k times. Solve the problem for each $k = 1, 2, \dots, m$ independently.

Reasoning Let's try to evaluate the function $f(s, k)$ that represents the maximum number of carrots all having size s that there can be after doing at most k cuts. It's quite intuitive that it's optimal to use all cuts at our disposal, the only configuration where not cutting is as good as cutting is when all carrots are of size 1. Which x 's should we choose for each cut c that we perform to evaluate $f(s, k)$. A strategy could be to choose $\forall_{1 \leq c \leq k} x_c = s$, let's take the maximum element m among all initial set, we can prove that with this strategy it takes $\lfloor \frac{m}{s} \rfloor$ to reduce all the carrots in pieces each of size of at most s . It's easy to see that we can do better, for example by organizing the cuts in this way $x_1 = 2^{k-1}s, x_2 = 2^{k-2}s, \dots, x_k = s$, let's take as example $s = 1$, we can see that is strictly unoptimal to choose any other cuts sequence, as it will take at least one more move to reach reach the same number of pieces. We can act as if we chose the first strategy (which is easier to implement) but we now know that the optimal number of moves to cut elements of initial size $\leq m$ in pieces of size $\leq s$ is $\log_2 \lfloor \frac{m}{s} \rfloor$. With careful implementation we can write a $\mathcal{O}(m \log m \log m)$ process that evaluates all $f(s, \forall c)$ in one sweep by checking multiples.

```
void solve()
{
    int n, m; cin >> n >> m;
    vector<int> a(n);
    for (int &e : a) cin >> e;
    sort(all(a));
    vector<int> cnt(m+1, 0);
    vector<ll> p(m+1, 0);
    for (int i = 0; i < n; i++) cnt[a[i]]++;
    for (int i = 1; i <= m; i++)
    {
        p[i] = p[i-1] + cnt[i];
    }

    vector<ll> ans(m+1, 0);
    for (int i = 1; i <= m; i++)
    {
        ll tot = cnt[i]; ans[0] = max(ans[0], tot);
        for (int j = i+1; j <= m; j += i)
        {
            int up = min(j-1+i, m), d = j-1;
```

```

    tot+=(p[m]-p[d])+(up%i==0)*cnt[up];
    int t=i, c=0;
    while(t<j)
    {
        t*=2;
        c++;
    }
    ans[c]=max(ans[c], tot);
    if(up==m) break;
}
}
ll mx=0;
for(int i=0; i<=m; i++)
{
    mx=max(ans[i], mx);
    if(i) cout<<mx<<" \n"[i==m];
}
}

```

Binary Bubble Sort Inversions

Source: Codeforces Round 1119 (1900)

Link: <https://codeforces.com/contest/2259/problem/F>

Problem Statement Define performing a *bubble* on an array $b_1, b_2, \dots, b_m$ as performing the following action:

- For each integer i ($1 \leq i < m$) in order, if $b_i > b_{i+1}$, swap b_i and b_{i+1} .

Define performing a *reverse bubble* on an array $b_1, b_2, \dots, b_m$ as performing the following action:

- For each integer i ($1 \leq i < m$) in reverse order, if $b_i > b_{i+1}$, swap b_i and b_{i+1} .

For example, if we perform a bubble on $[1, 1, 0]$, we transform it as follows: $[1, 1, 0] \rightarrow [1, 1, 0] \rightarrow [1, 0, 1]$. If we perform a reverse bubble on $[1, 1, 0]$, we transform it as follows: $[1, 1, 0] \rightarrow [1, 0, 1] \rightarrow [0, 1, 1]$. You are given an array $a_1, a_2, \dots, a_n$ consisting of only 0s and 1s and a binary string s of length n . You perform n operations in order, where the i -th operation performs a bubble on a if $s_i = 1$ and performs a reverse bubble on a if $s_i = 0$. For each of the $n + 1$ states of a — before any operations have been performed and after each operation — count the number of inversions in a .

Reasoning Let's try to simplify the two operations. Let's start with the bubble, let's scan the array from left to right the array and find the first 1 that we can find at position i , if no 1 is found we can assume the array is already sorted thus the number of inversions will remain 0. Notice now how not swapping an adjacent pair of 1, 1 can be seen as swapping an adjacent pair 1, 0, so we may as well assume that this operation grabs the first 1 at position i and takes it to the last position in the array, essentially we are doing this procedure $\forall_{n \geq j \geq i} \text{swap}(a_j, a_{j+1})$ in order from left to right. The reverse bubble works similarly, let's take the first zero at position h from the right and let's take it to position 1. Now let's see how the number of inversion changes for each operation, in case of the bubble let's take in consideration all the inversions the 1 at position i contributed to. This is exactly equal to the number of 0 present in the array at positions $[i, n]$. Symmetrically for the reverse bubble, the number of inversions the 0 at position h contributed to, are equal to the number of 1 present in the array at positions $[1, h]$. When an element is moved to the beginning or to the end it will never contribute to any inversion anymore, so we can see this as deleting it entirely from the array. We could implement a suffix 0 count array, and a prefix 1 count array as two segment trees where we can fastly delete the zero and the ones we move to the left and to the right. Run this simulation using one ascendingly sorted list of 1 positions, and one descendingly sorted list of 0 position, so at each operations we can swiftly recover the 1 or 0 to move.

```

struct segtree
{
    int n;

```

```

vector<int>seg;
segtree(int sz):n(sz),seg(2*n,0){}
void upd(int i,int v)
{
    seg[i+=n]+=v;
    for(int f=i>>1;f>=1;f>>=1)
    {
        seg[f]=seg[f<<1]+seg[f<<1|1];
    }
}
int que(int l,int r)
{
    int cum=0;
    for(int cl=l+n,cr=r+n;cl<=cr;cl>>=1,cr>>=1)
    {
        if(cl%2==1)cum+=seg[cl++];
        if(cr%2==0)cum+=seg[cr--];
    }
    return cum;
}
};
void solve()
{
    int n;cin>>n;
    vector<int>a(n);
    for(int &e:a)cin>>e;
    string s;cin>>s;
    int co=0;ll tot=0;
    for(int i=0;i<n;i++)
    {
        if(a[i]==1)co++;
        if(a[i]==0)tot+=co;
    }

    vector<int>poso,posz;
    segtree preo(n),sufz(n);
    for(int i=0;i<n;i++)
    {
        if(a[i]==1)
        {
            poso.pb(i);
            preo.upd(i,+1);
        }
        else
        {
            posz.pb(i);
            sufz.upd(i,+1);
        }
    }

    sort(all(poso));sort(all(posz));reverse(all(poso));
    cout<<tot<<' ';
    for(int i=0;i<n;i++)
    {
        if(s[i]=='0')
        {
            if(!posz.empty())
            {

```

```

        int t=posz.back();
        posz.ppb();
        tot-=preo.que(0,t);
        sufz.upd(t,-1);
    }
}
else if(s[i]=='1')
{
    if(!poso.empty())
    {
        int t=poso.back();
        poso.ppb();
        tot-=sufz.que(t,n-1);
        preo.upd(t,-1);
    }
}
cout<<tot<<" \n"[i==n-1];
}
}

```

Index Removal

Source: Codeforces Round 1119 (2000)

Link: <https://codeforces.com/contest/2259/problem/G>

Problem Statement An array $b_1, b_2, \dots, b_m$ is good if, for each i ($1 \leq i < m$), $b_{i+1} \geq b_i$ and $b_{i+1} - b_i \leq k$. An array of length 1 is always good. You are given an initially good array $a_1, a_2, \dots, a_n$. Solve the following problem for each i ($1 \leq i \leq n$) independently: The i -th element of a is removed, making $a = [a_1, a_2, \dots, a_{i-1}, a_{i+1}, \dots, a_n]$. In one operation, you are able to subtract 1 from any element of a . What is the minimum number of operations required to make a good again?

Reasoning Let's start by noticing that if we deleted the first or the last element then resulting array would still be good, therefore let's analyze only deletions of indices $i \in (1, n)$. Notice that when we delete an i in the aforementioned interval then a new adjacent pair appears, substituting two previous adjacent pairs $(i-1, i), (i, i+1) \rightarrow (i-1, i+1)$. Let's now analyze the new gap denoted as $N_i := a_{i+1} - a_{i-1}$. If $N_i \leq k$ then no adjustment has to be done, but if $N_i > k$ we know that we need to decrease a_{i+1} by $T_i := N_i - k$, this necessary action on a_{i+1} could compromise $a_{i+2} - a_{i+1}$ as it would be bigger by T_i . Let's be more meticulous and define $D_{i+1} := a_{i+2} - a_{i+1}$ before any operation, then if $k - D_{i+1} = 0$ then this T_i would propagate right causing $a_{i+2} := a_{i+2} - T_i$, if $k - D_{i+1} > 0$ then there could be two different cases:

- $T_i \leq k - D_{i+1}$ in this case there's no need to adjust a_{i+2} as there is space in D_{i+1} to absorb the T_i .
- $T_i > k - D_{i+1}$ in this case we need to decrease a_{i+2} by $T_{i+1} := T_i - k + D_{i+1}$. We can continue this argument recursively on the next adjacent pair until we get to a pair that absorbs the T_i coming from the previous pair, or there is no next pair to adjust.

What matters in this process are the spaces in all the D_i , let's denote it as $S_i := k - D_i$. If we deleted index i , generating a gap of T_i , then this would be reduced progressively $T_{i+1} := T_i - S_{i+1}$ and so on, til we reach the base cases described. Let's binary search the last difference that has to be adjusted, formally the last i s.t. $S_i < T_{i-1}$, which can be at most $i = n - 1$. Let's call it h_i (we can find it using a prefix sum array δ of the array S , to be more specific h_i is the last index such that $\delta_{h_i} - \delta_i > T_i$). Then we know that to find the number of operations which naively is $\sum_{i \leq j \leq h} T_j$, we may expand this in the following way:

$$\text{ans}_i := T_i + T_i(h_i - i) - \sum_{j=i+1}^{h_i} \sum_{k=i+1}^j S_k$$

At this point this is another prefix sum problem that can be solved with a prefix sum array Δ of the prefix sum array δ of $S := \{S_1, \dots, S_{n-1}\}$, yielding

$$\begin{aligned} \text{ans}_i &:= T_i + T_i(h_i - i) + \sum_{j=i+1}^{h_i} (\delta_j - \delta_i) \\ &= T_i + (T_i - \delta_i)(h_i - i) + \sum_{j=i+1}^{h_i} \delta_j \\ &= T_i + (T_i - \delta_i)(h_i - i) + \Delta_{h_i} - \Delta_i \end{aligned}$$

Notice how setting $h_i := i$ before the binary search, clears the edge case of $T_i \leq S_{i+1}$.

```
void solve()
{
    int n,k;cin>>n>>k;
    vector<int>a(n+1);
    vector<ll>p(n),pp(n),res(n+1,0);
    for(int i=1;i<=n;i++)cin>>a[i];
    for(int i=1;i<=n-1;i++)
    {
        p[i]=(k-a[i+1]+a[i])+p[i-1];
    }
    for(int i=1;i<=n-1;i++)
    {
        pp[i]=p[i]+pp[i-1];
    }
    for(int i=2;i<=n-1;i++)
    {
        int t=max(a[i+1]-a[i-1]-k,0);
        if(t!=0)
        {
            int lo=i+2,hi=n,ans=i+1;
            while(lo<=hi)
            {
                int mi=lo+(hi-lo)/2;
                if(p[mi-1]-p[i]<1ll*t)
                {
                    ans=mi;
                    lo=mi+1;
                }
                else
                {
                    hi=mi-1;
                }
            }
            res[i]=t+1ll*t*(ans-i-1)-(pp[ans-1]-pp[i])+1ll*(ans-i-1)*p[i];
        }
    }
    for(int i=1;i<=n;i++)cout<<res[i]<<" \n"[i==n];
}
```

Prefix Maximum Invariance

Source: Codeforces Round 1047 (1900)

Link: <https://codeforces.com/contest/2137/problem/F>

Problem Statement Given two arrays x and y both of size m , let z be another array of size m such that the prefix maximum at each position of z is the same as the prefix maximum at each position of x . Formally, $\max(x_1, x_2, \dots, x_i) = \max(z_1, z_2, \dots, z_i)$ should hold for all $1 \leq i \leq m$. Define $f(x, y)$ to be the maximum number of positions where $z_i = y_i$ over all possible arrays z .

You are given two sequences of integers a and b , both of size n . Please find the value of:

$$\sum_{l=1}^n \sum_{r=l}^n f([a_l, a_{l+1}, \dots, a_r], [b_l, b_{l+1}, \dots, b_r])$$

Reasoning Let's see fix a random subarray l, r we get $x := a[l : r]$ and $y := b[l : r]$ then how should we build z in order to maximize the matches with y ? Let's analyze x , by sweeping left to right each time we build the set I of prefix maximums positions, more formally we add index i to the set I if $x_i > x_j \ \forall_{j < i}$ holds. It's clear that $\forall_{i \in I} x_i = z_i$ has to hold, does this means that we can set any $z_j = y_j$ for all indices j not in I ? Not always let's take into consideration the following testcase

$$\begin{aligned} x &:= 5, 1, 2, 3, 4 \\ y &:= 1, 6, 1, 1, 1 \end{aligned}$$

In this case $I = \{1\}$, naturally $z_1 = 5$ but if we would set $z := 5, 6, 1, 1, 1$ this would break the prefix maximum property as $\max(x_1, x_2) = 5 \neq \max(z_1, z_2) = 6$. Notice that more elegantly we can set $z_i := y_i$ if and only if $x_i = y_i \vee \exists_{j < i} x_j \geq \max(x_i, y_i)$. Let's now suppose every element of every subarray is matching this would yield a total of $T := \sum_{l=1}^n l(n-l+1)$. Now for each index j , let's count the number of subarrays containing j s.t. we can set $z_i := y_i$, where i is the mapped j in the various x s, y s and z s. If $a_i = b_i$ it will be a match for all subarrays, else it can be a non match for some subarrays. Let's denote as $h_i < i$ be the closest index to the left of i such that $x_{h_i} \geq \max(x_i, y_i)$, in this case i will never be a match for all the subarrays that cross i and that start in the interval $(h_i, i]$. Therefore we subtract one for all matches that we supposed for i across all bad subarrays, yielding $T := T - (i - h_i)(n - i + 1)$. Let's sum up the contribution for each index in the following way:

$$\text{ans} := T - \sum_{i=1}^n [a_i \neq b_i] (i - h_i)(n - i + 1)$$

One again we can find all h_i s in many ways, personally i like segment trees as you may have noticed, but there is a simpler method that utilizes a monotonic stack. If $a_i, b_i \leq 1e9$ then the monotonic stack would be the only way.

```
struct segtree
{
    int n, def;
    vector<int> seg;
    segtree(int sz, int def): n(sz), def(def), seg(2*n, def) {}
    void upd(int i, int v)
    {
        seg[i+=n]=v;
        for(int f=i>>1; f>=1; f>>=1)
        {
            seg[f]=max(seg[f<<1], seg[f<<1|1]);
        }
    }
    int que(int l, int r)
    {
        int cum=def;
        for(int cl=l+n, cr=r+n; cl<=cr; cl>>=1, cr>>=1)
        {
            if(cl%2==1) cum=max(seg[cl++], cum);
            if(cr%2==0) cum=max(seg[cr--], cum);
        }
        return cum;
    }
};
```

```

    }
};

void solve()
{
    int n; cin >> n;
    vector<int> a(n), b(n);
    for(int &e: a) cin >> e;

    for(int &e: b) cin >> e;
    ll ans = 0;
    for(int l = 1; l <= n; l++)
    {
        ans += 1ll * (n - (l - 1)) * l;
    }

    segtree fmx(2 * n + 1, -1);
    for(int i = 0; i < n; i++)
    {
        if(a[i] != b[i])
        {
            int rma = fmx.que(b[i], 2 * n), rmb = fmx.que(a[i], 2 * n);
            ans -= 1ll * (n - i) * (i - min(rma, rmb));
        }
        fmx.upd(a[i], i);
    }
    cout << ans << '\n';
}

```

The Endspeaker (Easy Version)

Source: Codeforces Round 982 (1700)

<https://codeforces.com/contest/2027/problem/D1>

Problem Statement *This is the easy version of this problem. The only difference is that you only need to output the minimum total cost of operations in this version. You must solve both versions to be able to hack.*

You are given an array a of length n , and an array b of length m ($b_i > b_{i+1}$ for all $1 \leq i < m$). Initially, the value of k is 1. Your aim is to make the array a empty by performing one of these two operations repeatedly:

- **Type 1:** If $k < m$ and the array a is not empty, you can increase k by 1. This does not incur any cost.
- **Type 2:** You remove a non-empty prefix of array a such that its sum does not exceed b_k . This incurs a cost of $m - k$.

You need to minimize the total cost of the operations to make array a empty. If it is impossible to do this through any sequence of operations, output -1 . Otherwise, output the minimum total cost of the operations.

Reasoning Notice how deciding to delete some prefix from array a can be seen as some index i starting from jumping in a always from left to right, in the same way increasing k can be seen as some index j going from left to right in array b . Let's define the state $dp_{i,j}$ as the minimum cost to empty the array $a_j, \dots, a_n$ with $k = j$. If $i > n$ it means that the array is empty so $\forall_j dp_{i>n,j} := 0$, moreover any state $dp_{i,j}$ with $j > m$ is strictly forbidden therefore $dp_{i,j>m} := \infty$. Let's define the prefix sum array δ of the array a . Thanks to this we can write the following transition.

$$dp_{i,j} := \min dp_{i,j+1}, \min_{\forall h>j} [\delta_{h-1} - \delta_{i-1} \leq b_j] \infty + m - j + dp_{h,j}$$

This method would take $\mathcal{O}(n^2m)$, time complexity wise, which is still not good enough. Let's suppose from $dp_{i,j}$ we can jump to at most $dp_{h_m,j}$, where h_m is the rightmost index such that $\delta_{h_m-1} - \delta_{i-1} \leq b_j$ holds, jumping to any index $h < h_m$ is always unoptimal unless $j = m$, in this case it's as good as jumping to h_m . Therefore for each state $dp_{i,j}$ let's compute with a binary search $h_{i,j}$, reducing the time complexity to $\mathcal{O}(nm \log_2 n)$.

$$dp_{i,j} := \min dp_{i,j+1}, m - j + dp_{h_{i,j},j}$$

Finally pay attention when $h_{i,j} = i$ in this case set $dp_{i,j} = \infty$.

```
void solve()
{
    int n,m;cin>>n>>m;
    vector<int>a(n+1),b(m+1);
    vector<ll>p(n+1);
    vector<vector<ll>>dp(n+1,vector<ll>(m+1,-1));
    for(int i=1;i<=n;i++)cin>>a[i];
    for(int i=1;i<=m;i++)cin>>b[i];
    for(int i=1;i<=n;i++)p[i]=p[i-1]+a[i];
    auto dfs=[&](auto&& dfs,int i,int j)->ll
    {
        if(i>n)return 0;
        if(j>m)return IINF;
        if(dp[i][j]!=-1)
        {
            return dp[i][j];
        }
        if(b[j]<a[i])
        {
            return dp[i][j]=IINF;
        }
        ll mini=IINF;
        ll oa=dfs(dfs,i,j+1);
        if(oa!=IINF)
        {
            mini=min(oa,mini);
        }
        int lo=i,hi=n,ans=i;
        while(lo<=hi)
        {
            int mid=lo+(hi-lo)/2;
            if(p[mid]-p[i-1]<=b[j])
            {
                ans=mid;
                lo=mid+1;
            }
            else
            {
                hi=mid-1;
            }
        }
        ll ob=dfs(dfs,ans+1,j);
        if(ob!=IINF)
        {
            mini=min(m-j+ob,mini);
        }
        return dp[i][j]=mini;
    };

    ll ans=dfs(dfs,1,1);
    cout<<(ans==IINF?-1:ans)<<'\\n';
```

}

Unique Values (Hard Version)

Source: Codeforces Round 1093 (2000)

Link: <https://codeforces.com/contest/2219/problem/B2>

Problem Statement *The difference between the easy version and hard version is the maximum number of queries allowed. In this version, it is 33.*

There is a secret array a of length $2n + 1$, whose elements are integers from 1 to n . Each value appears exactly twice, except for one value, which appears exactly three times.

Your goal is to find the three positions of the value that appears three times.

To do this, you may ask the following query at most 33 times:

- Choose an integer k and an array s of k distinct indices between 1 and $2n + 1$.
- You will receive the number of different values among $a_{s_1}, a_{s_2}, \dots, a_{s_k}$ that appear exactly once — in other words, the count of values that are not repeated.

For example, if the values $a_{s_1}, a_{s_2}, \dots, a_{s_k}$ are $\{2, 1, 2, 3, 2, 3, 6, 7\}$, the answer to the query will be 3, as 1, 6, and 7 are the only values that appear rendering exactly once. The value 3 appears 2 times and 2 appears 3 times (which is more than once), so they are not counted.

Reasoning Notice one key property, if we give to the interactor a multiset of elements S such that for each $e \in S$ there are no more than 2 copies of e with response r , then $|S| - r$ is even (can be zero). This is quite trivial to prove, thus we'll leave this to the reader to confirm. What if there is exactly one element e in S such that there are three copies in S of the same e ? Then the inverse will hold so $|S| - r$ will be odd in this case. Thanks to this property we can binary search on the range $[1, 2n + 1]$ the index z of the rightmost third copy of e , indeed we can give to the interactor prefixes $1, 2, \dots, mid$, if $|S| - r$ is even then it means this prefix doesn't contain the last copy so we go right, else it contains it so we can look left. We can do the same with suffixes for the index x of the first leftmost copy in the range $[1, \dots, z]$, remember to manually add z to each suffix to give to the interactor. At this point we can binary search the last occurrence in the range (x, z) , i chose to use prefixes once again (remember again to add manually to this arrays the indeces x and z). It's really easy too see that we make less than 66 queries, why do we always make less then 33 though?

```
vector<int>hidden={};
int counteq(vector<int>&pos)
{
    map<int,int>x;
    for(int &i:pos)
    {
        x[hidden[i]]++;
    }
    int tot=0;
    for(const auto&[k,v]:x)if(v==1)tot++;
    cout<<"R: "<<tot<<'\n';
    return tot;
}

int ask(vector<int>&pos)
{
    cout<<"? "<<sz(pos)<<' ';
    for(int &p:pos)cout<<p<<' ';
    cout<<endl;
```

```

    int k;
    if(!hidden.empty())
    {
        k=counteq(pos);
    }
    else
    {
        cin>>k;
    }
    return k;
}
void solve()
{
    int n;cin>>n;
    int x=-1,y=-1,z=-1;
    int lo=1,hi=2*n+1;
    while(lo<=hi)
    {
        int mi=lo+(hi-lo)/2;
        vector<int>p;
        for(int i=1;i<=mi;i++)
        {
            p.pb(i);
        }
        if((sz(p)-ask(p))%2!=0)
        {
            z=mi;
            hi=mi-1;
        }
        else
        {
            lo=mi+1;
        }
    }
    lo=1,hi=z-1;
    while(lo<=hi)
    {
        int mi=lo+(hi-lo)/2;
        vector<int>s;
        for(int i=z;i>=mi;i--)
        {
            s.pb(i);
        }
        if((sz(s)-ask(s))%2!=0)
        {
            x=mi;
            lo=mi+1;
        }
        else
        {
            hi=mi-1;
        }
    }
    lo=x+1,hi=z-1;
    while(lo<=hi)
    {
        int mi=lo+(hi-lo)/2;
        vector<int>in{x,z};
    }
}

```

```
    for(int i=x+1;i<=mi;i++)
    {
        in.pb(i);
    }
    if((sz(in)-ask(in))%2!=0)
    {
        y=mi;
        hi=mi-1;
    }
    else
    {
        lo=mi+1;
    }
}
cout<<"! "<<x<<' '<<y<<' '<<z<<endl;
}
```

I included the local checker so it's easier to test, also it's good practice to do in contest if it's easy to implement (it saves a lot of time).